

Machine Learning Setup For Natural Situation

Dr. Sujit Das
Department CSE-AI&ML

1 A simple machine learning setup

We have seen some ideal conditions in the case of Boolean function realization using Neural Networks (NNs). What we have observed is that there exists a very clear segregation of data, which helps us draw a decision boundary that clearly divides all the inputs into two classes.

However, this is not a representation of real-world problems.

In real-world problems, such perfect segregation is usually not possible. We will always have some error, but our objective is to minimize or correct this error as much as possible.

Typical Machine Learning Setup

Data

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$$

where

- x_i : Input feature vector
- y_i : Desired output (target/label)

Assume there exists a true relationship between x and y :

$$y = f(x, w)$$

where

- $f(\cdot)$: Unknown underlying function
- w : Model parameters (weights)

The machine learning model attempts to estimate this relationship.
The predicted output is

$$\hat{y} = \hat{f}(x, \hat{w})$$

where

- $\hat{f}$: Estimated model
- $\hat{w}$: Learned (estimated) parameter vector

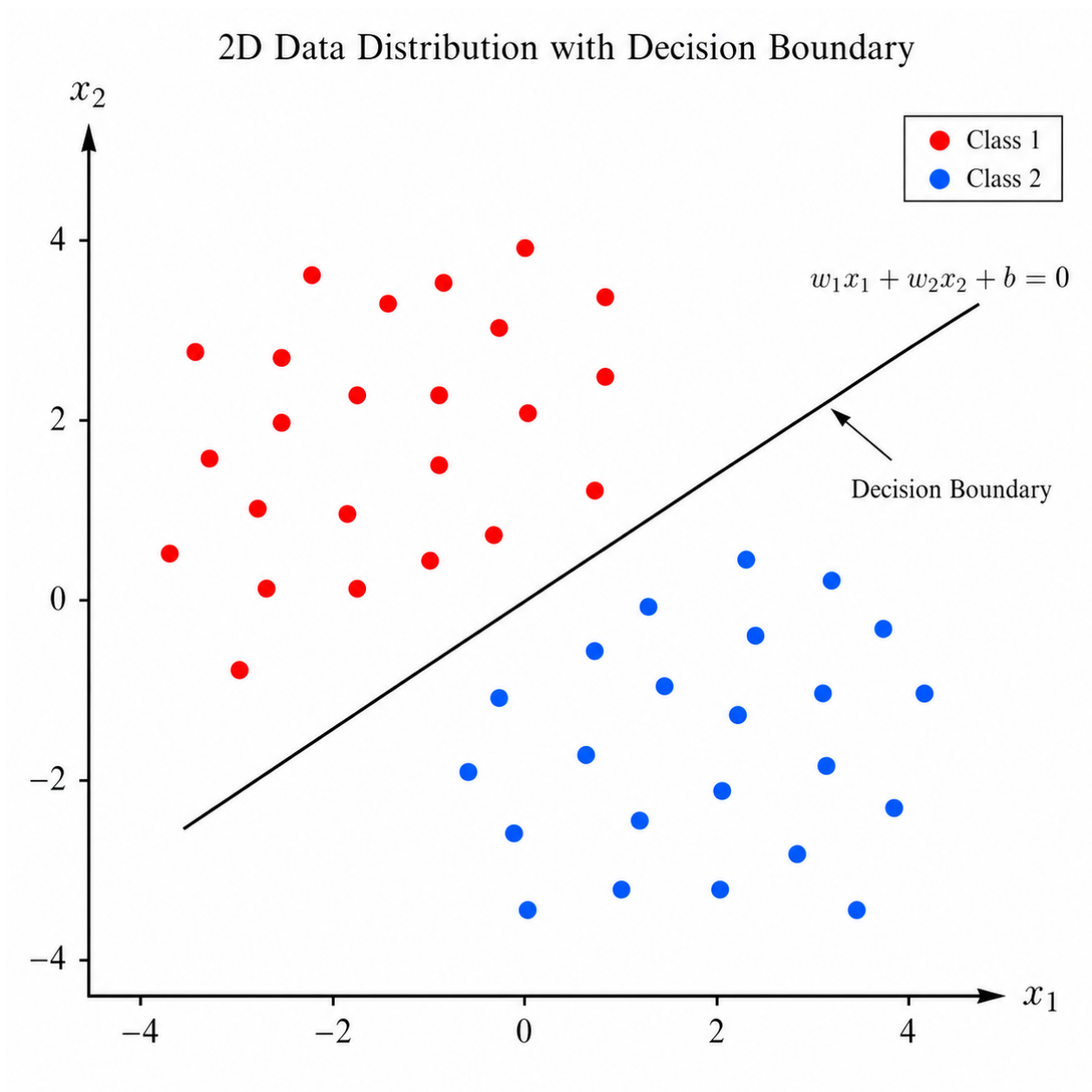


Figure 1: A simple binary Classification

Learning Objective

Find the parameter vector $\hat{w}$ such that the prediction error is minimized:

$$\hat{w} = \arg \min_w \sum_{i=1}^n L(y_i, f(x_i, w))$$

where

- $L(\cdot)$: Loss function measuring the discrepancy between the true output and the predicted output.
- y_i : True (target) output for the i^{th} sample.
- $f(x_i, w)$: Model prediction for the i^{th} sample.
- w : Parameter (weight) vector of the model.
- $\hat{w}$: Optimal parameter vector that minimizes the total loss.

The corresponding prediction using the learned parameters is

$$\hat{y} = \hat{f}(x, \hat{w})$$

where $\hat{w}$ is obtained through the optimization process above.

Model

The model output is given by the sigmoid activation function:

$$\hat{y} = \frac{1}{1 + e^{-(w^T x)}}$$

where

$$z = w^T x$$

and therefore

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The linear combination of inputs is

$$w^T x = \sum_{i=1}^n w_i x_i$$

where

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

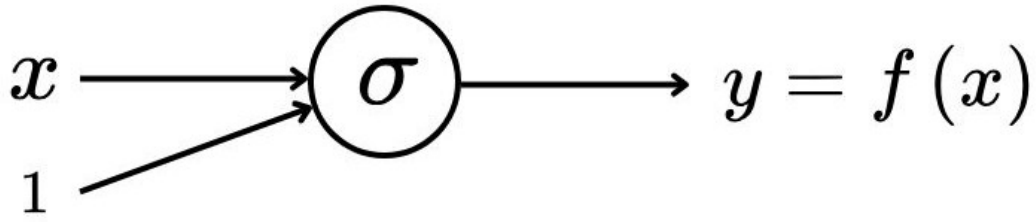


Figure 2: Simple Model

Parameters

$$w = \{w_1, w_2, \dots, w_n\}$$

Learning Algorithm The parameters w are learned using the Gradient Descent algorithm.

With bias b the model will become:

$$\hat{y} = \frac{1}{1 + e^{-(w^T x + b)}}$$

Recalling the learning objective function:

$$L(\hat{w}) = \sum_{i=1}^n L(y_i, f(x_i, w))$$

we are simplifying our model with only one input Input for the training $\{x_i, y_i\}_i \rightarrow$
N pairs of x,y

So that the training objective becomes

$$\arg \min_{w,b} L(w, b) = \sum_{i=1}^n L((y_i - f(x_i))^2)$$

Suppose we consider two points

$(x, y) = (0.5, 0.2)$ **and** $(2.5, 0.9)$ then $f(0.5) = 0.2$ and $f(2.5) = 0.9$. Let the initial weights $w = 0.5, b = 0$, then

Loss Function

For a training dataset containing N samples, the Sum of Squared Error (SSE) loss is defined as

$$L(w, b) = \frac{1}{2} \sum_{i=1}^N (y_i - f(m_i))^2$$

For $N = 2$, the loss becomes

$$L(w, b) = \frac{1}{2} \left[(y_1 - f(m_1))^2 + (y_2 - f(m_2))^2 \right]$$

Given

$$y_1 = 0.9, \quad m_1 = 2.0$$

$$y_2 = 0.2, \quad m_2 = 0.5$$

and the sigmoid activation function

$$f(m) = \frac{1}{1 + e^{-m}}$$

the loss can be written as

$$L(w, b) = \frac{1}{2} \left[(0.9 - f(2.0))^2 + (0.2 - f(0.5))^2 \right]$$

Substituting the sigmoid values,

$$f(2.0) = \frac{1}{1 + e^{-2.0}} \approx 0.8808$$

$$f(0.5) = \frac{1}{1 + e^{-0.5}} \approx 0.6225$$

Therefore,

$$\begin{aligned} L(w, b) &= \frac{1}{2} \left[(0.9 - 0.8808)^2 + (0.2 - 0.6225)^2 \right] \\ &= \frac{1}{2} [0.00037 + 0.17850] \end{aligned}$$

$$L(w, b) \approx 0.0894$$

Hence, the loss value for the given samples is

$$\boxed{L(w, b) \approx 0.0894}$$

Table 1: Loss values for different parameter settings

w	b	$L(w, b)$
0.50	0.00	0.0730
-0.10	0.00	0.1481
0.94	-0.94	0.0214
1.42	-1.77	0.0028
1.65	-2.08	0.0003
1.76	-2.27	0.0000

Error Surface and Gradient Descent Path

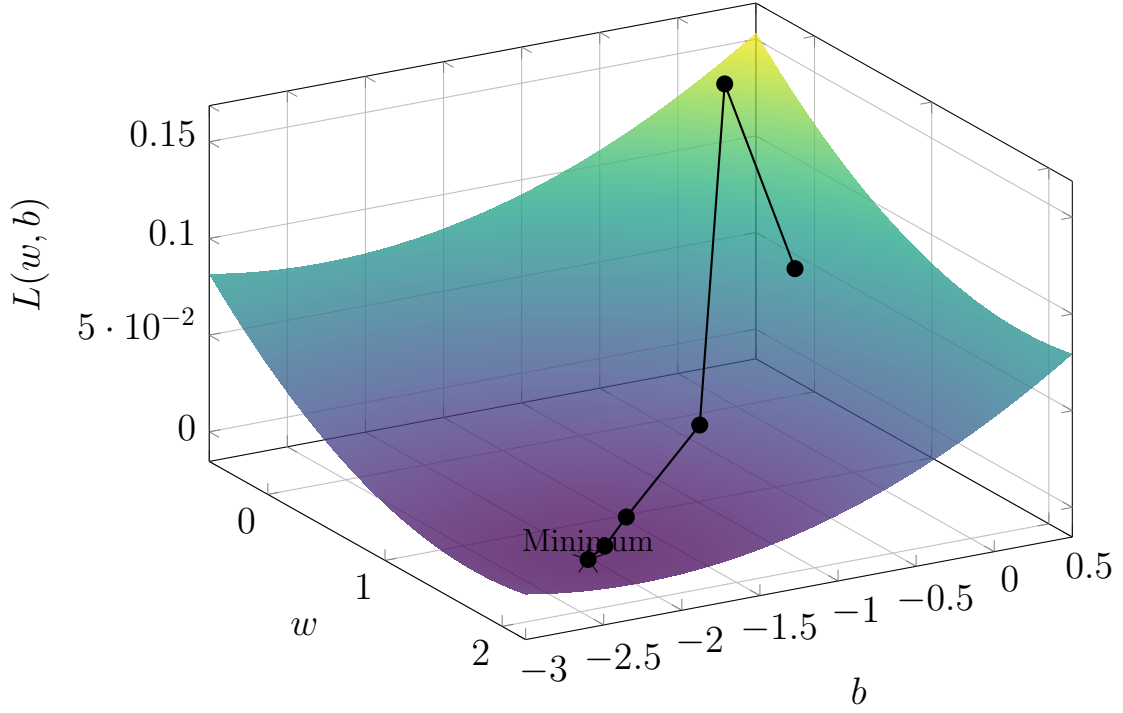


Figure 3: Smooth Error Surface with Gradient Descent Updates

1.1 Finding Parameters: A Systematic Way to Learn

Suppose the parameter vector is

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix}$$

and the update direction is

$$\Delta\theta = \begin{bmatrix} \Delta w \\ \Delta b \end{bmatrix}.$$

The updated parameter vector is

$$\theta_{\text{new}} = \theta + \eta\Delta\theta,$$

where η is the learning rate.

Taylor Series Expansion Assume

$$\Delta\theta = u.$$

Then, using Taylor series,

$$L(\theta + \eta u) = L(\theta) + \eta u^T \nabla L(\theta) + \frac{\eta^2}{2} u^T \nabla^2 L(\theta) u + O(\eta^3).$$

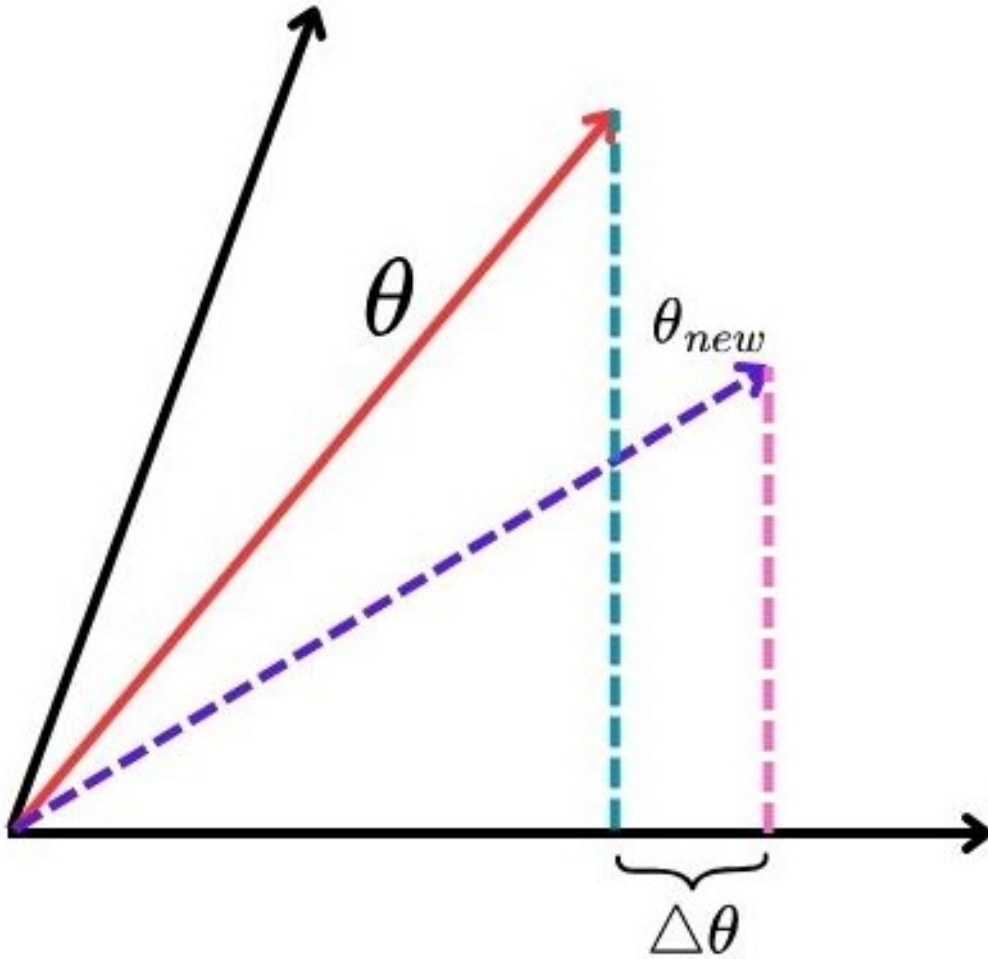


Figure 4: Theta Graph

Example For a one-dimensional loss function

$$L(w) = w^2,$$

the derivative is

$$\frac{dL(w)}{dw} = 2w.$$

For a two-parameter loss function

$$L(w, b) = w^2 + b^2,$$

the partial derivatives are

$$\frac{\partial L(w, b)}{\partial w} = 2w,$$

$$\frac{\partial L(w, b)}{\partial b} = 2b.$$

Gradient Vector The gradient of the loss function is

$$\nabla L(\theta) = \begin{bmatrix} \frac{\partial L(w, b)}{\partial w} \\ \frac{\partial L(w, b)}{\partial b} \end{bmatrix}.$$

Hessian Matrix The Hessian matrix is

$$\nabla^2 L(\theta) = \begin{bmatrix} \frac{\partial^2 L(w, b)}{\partial w^2} & \frac{\partial^2 L(w, b)}{\partial w \partial b} \\ \frac{\partial^2 L(w, b)}{\partial b \partial w} & \frac{\partial^2 L(w, b)}{\partial b^2} \end{bmatrix}.$$

Descent Condition To decrease the loss, we require

$$L(\theta + \eta u) - L(\theta) < 0.$$

Hence, the update direction should be chosen opposite to the gradient:

$$u = -\nabla L(\theta).$$

This leads to the Gradient Descent update rule

$$\theta_{\text{new}} = \theta - \eta \nabla L(\theta).$$

Choosing the Descent Direction

From the Taylor series expansion,

$$L(\theta + \eta u) - L(\theta) = \eta u^T \nabla L(\theta).$$

For the loss to decrease, we require

$$L(\theta + \eta u) - L(\theta) < 0.$$

Therefore,

$$u^T \nabla L(\theta) < 0.$$

Let β be the angle between u and $\nabla L(\theta)$. Then

$$\cos(\beta) = \frac{u^T \nabla L(\theta)}{\|u\| \|\nabla L(\theta)\|}.$$

Since

$$-1 \leq \cos(\beta) \leq 1,$$

let

$$K = \|u\| \|\nabla L(\theta)\|.$$

Then

$$-K \leq u^T \nabla L(\theta) = K \cos(\beta) \leq K.$$

The loss decreases most rapidly when

$$u^T \nabla L(\theta)$$

is as negative as possible.

This occurs when

$$\beta = 180^\circ,$$

because

$$\cos(180^\circ) = -1.$$

Hence, the optimal descent direction is

$$u = -\nabla L(\theta).$$

In other words, we move in the direction opposite to the gradient.

Gradient Descent Parameter Update

The parameter update rules are

$$w_{t+1} = w_t - \eta \nabla w_t,$$

$$b_{t+1} = b_t - \eta \nabla b_t.$$

where

$$\nabla w_t = \left. \frac{\partial L(w, b)}{\partial w} \right|_{w=w_t, b=b_t},$$

and

$$\nabla b_t = \left. \frac{\partial L(w, b)}{\partial b} \right|_{w=w_t, b=b_t}.$$

In vector form,

$$\theta_{t+1} = \theta_t + \eta (-\nabla L(\theta_t)),$$

or equivalently,

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)}.$$

Gradient Computation for a Single Neuron

Consider a single neuron with input x , target y , and sigmoid activation as shown in the Figure 1

$$f(n) = \frac{1}{1 + e^{-(wx+b)}},$$

where

$$n = wx + b.$$

The loss function for a single training sample is

$$L(w, b) = \frac{1}{2} \left(f(n) - y \right)^2.$$

The gradient with respect to w is

$$\nabla_w = \frac{\partial L}{\partial w} = \frac{\partial}{\partial w} \left[\frac{1}{2} \left(f(n) - y \right)^2 \right].$$

Applying the chain rule,

$$\nabla_w = \left(f(n) - y \right) \frac{\partial f(n)}{\partial w}.$$

Since

$$f(n) = \frac{1}{1 + e^{-(wx+b)}},$$

we obtain

$$\frac{\partial f(n)}{\partial w} = \frac{\partial f(n)}{\partial n} \frac{\partial n}{\partial w}.$$

Now,

$$\frac{\partial n}{\partial w} = x,$$

and

We compute the derivative with respect to w :

$$\frac{\partial}{\partial w} \left(\frac{1}{1 + e^{-(wx+b)}} \right)$$

Using the chain rule,

$$\begin{aligned} &= -\frac{1}{\left(1 + e^{-(wx+b)}\right)^2} \cdot \frac{\partial}{\partial w} \left(e^{-(wx+b)} \right) \\ &= -\frac{1}{\left(1 + e^{-(wx+b)}\right)^2} \cdot e^{-(wx+b)} \cdot \frac{\partial}{\partial w} \left(-(wx + b) \right) \end{aligned}$$

$$\begin{aligned}
&= -\frac{1}{\left(1 + e^{-(wx+b)}\right)^2} \cdot e^{-(wx+b)} \cdot (-x) \\
&= \frac{e^{-(wx+b)}}{\left(1 + e^{-(wx+b)}\right)^2} \cdot x
\end{aligned}$$

Multiplying and dividing by $\left(1 + e^{-(wx+b)}\right)$,

$$= \frac{1}{1 + e^{-(wx+b)}} \cdot \frac{e^{-(wx+b)}}{1 + e^{-(wx+b)}} \cdot x$$

Since

$$f(n) = \frac{1}{1 + e^{-(wx+b)}},$$

and

$$1 - f(n) = \frac{e^{-(wx+b)}}{1 + e^{-(wx+b)}},$$

we obtain

$$\frac{\partial f(n)}{\partial w} = f(n)(1 - f(n))x.$$

Similarly,

$$\frac{\partial f(n)}{\partial b} = f(n)(1 - f(n)).$$

Therefore, the derivative of the sigmoid activation is

$$\boxed{\frac{\partial f(n)}{\partial n} = f(n)(1 - f(n))}$$

and

$$\boxed{\frac{\partial f(n)}{\partial w} = f(n)(1 - f(n))x}.$$

$$\frac{\partial f(n)}{\partial n} = f(n)(1 - f(n)).$$

Therefore,

$$\frac{\partial f(n)}{\partial w} = f(n)(1 - f(n))x.$$

Substituting into the gradient expression,

$$\nabla_w = \left(f(n) - y\right)f(n)(1 - f(n))x.$$

Similarly,

$$\nabla_b = \frac{\partial L}{\partial b} = \left(f(n) - y\right)f(n)\left(1 - f(n)\right).$$

Gradient for Multiple Training Samples

For a dataset

$$\{(x_i, y_i)\}_{i=1}^N,$$

the gradient with respect to w is

$$\nabla_w = \sum_{i=1}^N \left(f(n_i) - y_i\right)f(n_i)\left(1 - f(n_i)\right)x_i,$$

where

$$n_i = wx_i + b.$$

Similarly,

$$\nabla_b = \sum_{i=1}^N \left(f(n_i) - y_i\right)f(n_i)\left(1 - f(n_i)\right).$$

The parameter update equations become

$$w_{t+1} = w_t - \eta \nabla_w,$$

$$b_{t+1} = b_t - \eta \nabla_b.$$

Difference between MLP and FFN

Table 2: Difference Between MLP and FFN

Property	MLP (Multi-Layer Perceptron)	FFN (Feed-Forward Network)
Terminology	Emphasizes the presence of multiple layers (at least one hidden layer).	Broader term describing any neural network in which information flows only in the forward direction without loops or feedback connections.
Depth Requirement	Must contain one or more hidden layers.	May include a single-layer model (e.g., logistic regression) or a deep network.
Usage Context	Commonly used in classical neural networks and early ANN literature.	Widely used in modern deep learning as a general architectural term.
Architecture	Typically consists of fully connected (dense) layers with nonlinear activation functions.	Can refer to any feed-forward architecture, including dense, convolutional, or other forward-only structures.
Modern Meaning	Often refers specifically to a stack of dense hidden layers.	Often used as a generic building block or submodule within larger neural network architectures.
Relationship	Every MLP is a Feed-Forward Network.	Not every Feed-Forward Network is necessarily an MLP.

2 Multi-Layer Neural Networks and Backpropagation

Consider a neural network with L layers.

For layer i ,

$$W_i \in \mathbb{R}^{n_i \times n_{i-1}},$$

$$b_i \in \mathbb{R}^{n_i}.$$

For the $(L - 1)^{th}$ hidden layer, let there be k neurons corresponding to k output classes.

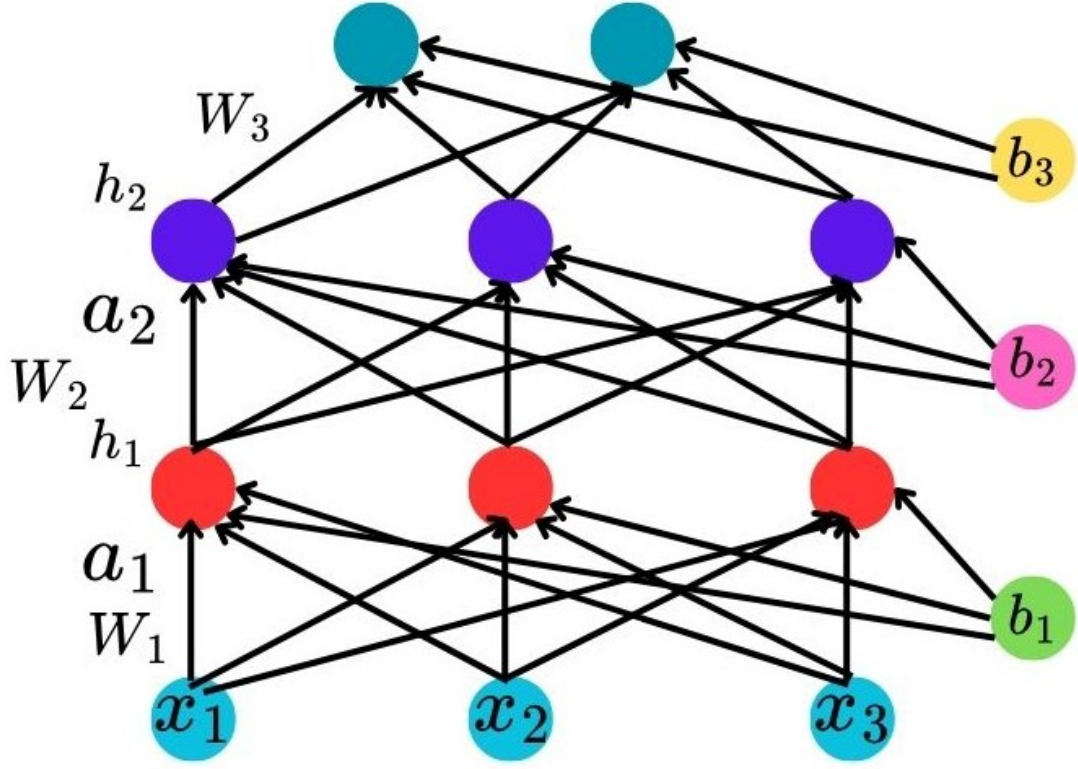


Figure 5: A simple neural Network for simulation for Backpropagation

Forward Propagation

The pre-activation vector at layer i is

$$a_i(x) = b_i + W_i h_{i-1}(x).$$

In matrix form,

$$\begin{bmatrix} a_{i1} \\ a_{i2} \\ \vdots \\ a_{in_i} \end{bmatrix} = \begin{bmatrix} b_{i1} \\ b_{i2} \\ \vdots \\ b_{in_i} \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n_{i-1}} \\ w_{21} & w_{22} & \cdots & w_{2n_{i-1}} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n_i1} & w_{n_i2} & \cdots & w_{n_i n_{i-1}} \end{bmatrix} \begin{bmatrix} h_{i-1,1} \\ h_{i-1,2} \\ \vdots \\ h_{i-1,n_{i-1}} \end{bmatrix}.$$

The hidden representation is obtained by applying an element-wise activation function:

$$h_i = g(a_i).$$

The network output is

$$f(x) = h_L(x) = O(a_L(x)),$$

where $O(\cdot)$ denotes the output activation function.

Training Data

Given

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N.$$

The model prediction is

$$\hat{y}_i = f(x_i).$$

For a three-layer network,

$$\hat{y}_i = O(W_3 g(W_2 g(W_1 x_i + b_1) + b_2) + b_3).$$

Model Parameters

The parameter set is

$$\theta = \{W_1, \dots, W_L, b_1, \dots, b_L\}.$$

Learning Algorithm

The parameters are learned using

Gradient Descent with Backpropagation.

Output Layer Choices

Task	Output Activation	Loss Function
Regression	Linear	Squared Error
Binary Classification	Sigmoid	Cross Entropy
Multi-class Classification	Softmax	Cross Entropy

Backpropagation

Two important questions need to be answered:

1. How should the loss function $L(\theta)$ be chosen?
2. How can the gradient $\nabla_{\theta} L$ be computed efficiently?

The gradient vector consists of

$$\nabla_{\theta} L = [\nabla_{W_1} L, \nabla_{W_2} L, \dots, \nabla_{W_L} L, \nabla_{b_1} L, \nabla_{b_2} L, \dots, \nabla_{b_L} L].$$

For classification with softmax output,

$$\hat{y} = g(a_L),$$

and the cross-entropy loss is

$$L(\theta) = - \sum_{c=1}^k y_c \log(\hat{y}_c).$$

Gradient Descent Algorithm

Initialize

$$\theta_0.$$

For $t = 0, 1, 2, \dots$,

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t).$$

Repeat until convergence or the maximum number of iterations is reached.

Computing Individual Gradients

For example, to update a weight w_{112} ,

$$\frac{\partial L(\theta)}{\partial w_{111}}$$

must be computed.

Backpropagation efficiently computes all such partial derivatives using the chain rule, propagating gradients from the output layer toward the input layer.

Backpropagation Using the Chain Rule

Consider a simple neural network as in Figure 2 and suppose we want to compute

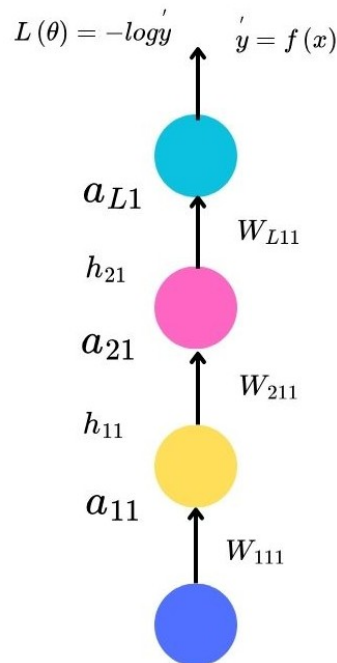


Figure 6: A simple thin network

$$\frac{\partial L(\theta)}{\partial w_{111}}.$$

Using the chain rule,

$$\frac{\partial L(\theta)}{\partial w_{111}} = \frac{\partial L(\theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_{L1}} \cdot \frac{\partial a_{L1}}{\partial h_{L-1,1}} \cdot \frac{\partial h_{L-1,1}}{\partial a_{L-1,1}} \cdots \frac{\partial a_{11}}{\partial w_{111}}.$$

For convenience, we may write

$$\frac{\partial L(\theta)}{\partial w_{111}} = \frac{\partial L(\theta)}{\partial h_{11}} \cdot \frac{\partial h_{11}}{\partial w_{111}}.$$

The network output is

$$\hat{y} = f(x) = O(W_L h_{L-1} + b_L).$$

Every weight and bias contributes to the final prediction and therefore contributes to the loss.

Instead of computing the derivatives independently, we propagate gradients backward through the network using the chain rule.

For a three-layer network,

$$\frac{\partial L(\theta)}{\partial w_{111}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_3} \cdot \frac{\partial a_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{111}}.$$

This process is called **Backpropagation**.

Meaning of a Derivative

A derivative measures the sensitivity of one quantity with respect to another.

For example,

$$\frac{\partial a_1}{\partial w_{111}}$$

indicates how much the activation a_1 changes when the weight w_{111} is changed slightly.

Similarly,

$$\frac{\partial L}{\partial w}$$

measures how much the loss changes when the weight w changes.

Quantities of Interest

In backpropagation we compute:

- Gradient with respect to output units.
- Gradient with respect to hidden units.
- Gradient with respect to weights and biases.

These gradients are then used to update the parameters through gradient descent.

Loss Functions

Depending on the task, different loss functions can be used:

- Squared Error Loss

$$L(\theta) = \frac{1}{2} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- Cross-Entropy Loss

$$L(\theta) = - \sum_{i=1}^N y_i \log(\hat{y}_i)$$

- Softmax Cross-Entropy Loss

$$L(\theta) = - \sum_{i=1}^N \sum_{c=1}^K y_{ic} \log(\hat{y}_{ic})$$

Derivative of Softmax Cross-Entropy Loss

Assume that we are interested in computing gradients with respect to the output layer biases and weights. Instead of considering a single weight or bias, we consider the derivative with respect to the entire output vector.

Let the network output be

$$\hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_K \end{bmatrix},$$

where K is the number of output classes.

For a training sample belonging to class ℓ , the cross-entropy loss is

$$L(\theta) = -\log(\hat{y}_\ell).$$

Equivalently,

$$L(\theta) = - \sum_{i=1}^K e_i^{(\ell)} \log(\hat{y}_i),$$

where

$$e^{(\ell)} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

is the one-hot target vector whose ℓ^{th} entry is equal to 1.

Derivative with respect to the output probabilities The derivative of the loss with respect to $\hat{y}_i$ is

$$\frac{\partial L(\theta)}{\partial \hat{y}_i} = \begin{cases} -\frac{1}{\hat{y}_\ell}, & i = \ell, \\ 0, & i \neq \ell. \end{cases}$$

Using indicator notation,

$$\frac{\partial L(\theta)}{\partial \hat{y}_i} = -\frac{\mathbb{I}(i = \ell)}{\hat{y}_\ell}.$$

Hence,

$$\nabla_{\hat{\mathbf{y}}} L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial \hat{y}_1} \\ \frac{\partial L}{\partial \hat{y}_2} \\ \vdots \\ \frac{\partial L}{\partial \hat{y}_K} \end{bmatrix} = -\frac{1}{\hat{y}_\ell} e^{(\ell)}.$$

Softmax Output Layer Let

$$\mathbf{a}_L = \begin{bmatrix} a_{L1} \\ a_{L2} \\ \vdots \\ a_{LK} \end{bmatrix}$$

be the pre-activation vector of the output layer.
The softmax output is

$$\hat{y}_i = \frac{\exp(a_{Li})}{\sum_{j=1}^K \exp(a_{Lj})}.$$

We are interested in

$$\frac{\partial L(\theta)}{\partial a_{Li}}.$$

Using the chain rule,

$$\frac{\partial L(\theta)}{\partial a_{Li}} = \sum_{j=1}^K \frac{\partial L(\theta)}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial a_{Li}}.$$

We are interested in computing the derivative of the loss function with respect to the pre-activation outputs a_{Li} of the output layer.

Using the chain rule,

$$\frac{\partial L(\theta)}{\partial a_{Li}} = \frac{\partial(-\log \hat{y}_\ell)}{\partial \hat{y}_\ell} \cdot \frac{\partial \hat{y}_\ell}{\partial a_{Li}}.$$

Since

$$\hat{y}_\ell = \frac{\exp(a_{L\ell})}{\sum_{j=1}^K \exp(a_{Lj})},$$

the derivative of the loss with respect to $\hat{y}_\ell$ is

$$\frac{\partial(-\log \hat{y}_\ell)}{\partial \hat{y}_\ell} = -\frac{1}{\hat{y}_\ell}.$$

Therefore,

$$\frac{\partial L(\theta)}{\partial a_{Li}} = -\frac{1}{\hat{y}_\ell} \frac{\partial}{\partial a_{Li}} \left(\frac{\exp(a_{L\ell})}{\sum_{j=1}^K \exp(a_{Lj})} \right).$$

Applying the quotient rule,

$$= -\frac{1}{\hat{y}_\ell} \left(\frac{\delta_{i\ell} \exp(a_{L\ell}) \sum_{j=1}^K \exp(a_{Lj}) - \exp(a_{L\ell}) \frac{\partial}{\partial a_{Li}} \left(\sum_{j=1}^K \exp(a_{Lj}) \right)}{\left(\sum_{j=1}^K \exp(a_{Lj}) \right)^2} \right),$$

where

$$\delta_{i\ell} = \begin{cases} 1, & i = \ell, \\ 0, & i \neq \ell. \end{cases}$$

Since

$$\frac{\partial}{\partial a_{Li}} \left(\sum_{j=1}^K \exp(a_{Lj}) \right) = \exp(a_{Li}),$$

we obtain

$$\frac{\partial L(\theta)}{\partial a_{Li}} = -\frac{1}{\hat{y}_\ell} \left(\frac{\delta_{i\ell} \exp(a_{L\ell})}{\sum_{j=1}^K \exp(a_{Lj})} - \frac{\exp(a_{L\ell}) \exp(a_{Li})}{\left(\sum_{j=1}^K \exp(a_{Lj}) \right)^2} \right).$$

Recognizing the softmax probabilities,

$$\hat{y}_\ell = \frac{\exp(a_{L\ell})}{\sum_{j=1}^K \exp(a_{Lj})},$$

and

$$\hat{y}_i = \frac{\exp(a_{Li})}{\sum_{j=1}^K \exp(a_{Lj})},$$

gives

$$\frac{\partial L(\theta)}{\partial a_{Li}} = -\frac{1}{\hat{y}_\ell} (\delta_{i\ell} \hat{y}_\ell - \hat{y}_\ell \hat{y}_i).$$

Simplifying,

$$\frac{\partial L(\theta)}{\partial a_{Li}} = -(\delta_{i\ell} - \hat{y}_i).$$

Hence,

$$\boxed{\frac{\partial L(\theta)}{\partial a_{Li}} = \hat{y}_i - \delta_{i\ell}}$$

for all $i = 1, \dots, K$.

Let

$$e^{(\ell)} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

be the one-hot vector corresponding to class ℓ .

Then the gradient vector with respect to the output layer pre-activations is

$$\nabla_{a_L} L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial a_{L1}} \\ \frac{\partial L}{\partial a_{L2}} \\ \vdots \\ \frac{\partial L}{\partial a_{LK}} \end{bmatrix} = \begin{bmatrix} -(\mathbb{I}_{\ell=1} - \hat{y}_1) \\ -(\mathbb{I}_{\ell=2} - \hat{y}_2) \\ \vdots \\ -(\mathbb{I}_{\ell=K} - \hat{y}_K) \end{bmatrix}.$$

Equivalently,

$$\boxed{\nabla_{a_L} L(\theta) = \hat{\mathbf{y}} - e^{(\ell)}}$$

6

which is the standard backpropagation error term at the output layer for a Softmax-Cross-Entropy model.

2.1 Gradient with Respect to Hidden Layers

To understand backpropagation through hidden layers, consider the computational graph shown in Figure 7. Let

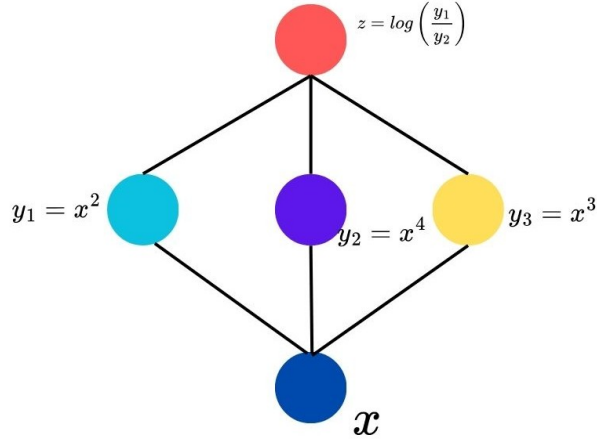


Figure 7: Example computational graph illustrating multiple paths from input x to output z .

$$y_1 = x^2, \quad y_2 = x^4, \quad y_3 = x^3,$$

and

$$z = \log\left(\frac{y_1}{y_2}\right).$$

Unlike a simple chain structure where only one path exists between variables, here the variable x influences the output through multiple paths. Therefore, the ordinary chain rule is not sufficient. We must use the multivariable chain rule.

The derivative of z with respect to x is obtained by summing contributions from all paths:

$$\frac{dz}{dx} = \sum_{i=1}^3 \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x}.$$

In general, if a function $p(z)$ depends on several intermediate quantities

$$q_1(z), q_2(z), \dots, q_m(z),$$

then the multivariable chain rule states that

$$\frac{\partial p(z)}{\partial z} = \sum_m \frac{\partial p(z)}{\partial q_m(z)} \frac{\partial q_m(z)}{\partial z}.$$

This principle forms the foundation of backpropagation in neural networks.

Suppose we are interested in computing the gradient of the loss function with respect to a hidden unit h_{ij} :

$$\frac{\partial L(\theta)}{\partial h_{ij}}.$$

The hidden unit h_{ij} affects all pre-activation values in the next layer,

$$a_{i+1,1}, a_{i+1,2}, \dots, a_{i+1,K}.$$

Applying the multivariable chain rule,

$$\frac{\partial L(\theta)}{\partial h_{ij}} = \sum_{m=1}^K \frac{\partial L(\theta)}{\partial a_{i+1,m}} \frac{\partial a_{i+1,m}}{\partial h_{ij}}.$$

Recall that the pre-activation values are computed as

$$a_{i+1} = W_{i+1}h_i + b_{i+1}.$$

For example,

$$\begin{bmatrix} a_{31} \\ a_{32} \end{bmatrix} = \begin{bmatrix} w_{311} & w_{312} & w_{313} \\ w_{321} & w_{322} & w_{323} \end{bmatrix} \begin{bmatrix} h_{21} \\ h_{22} \\ h_{23} \end{bmatrix} + \begin{bmatrix} b_{31} \\ b_{32} \end{bmatrix}.$$

Hence,

$$a_{31} = w_{311}h_{21} + w_{312}h_{22} + w_{313}h_{23} + b_{31},$$

and therefore,

$$\frac{\partial a_{31}}{\partial h_{22}} = w_{312}.$$

More generally,

$$\frac{\partial a_{i+1,m}}{\partial h_{ij}} = w_{i+1,m,j}.$$

Substituting into the chain rule,

$$\frac{\partial L(\theta)}{\partial h_{ij}} = \sum_{m=1}^K \frac{\partial L(\theta)}{\partial a_{i+1,m}} w_{i+1,m,j}.$$

Define the gradient vector with respect to the pre-activation units of layer $i + 1$:

$$\nabla_{a_{i+1}} L(\theta) = \begin{bmatrix} \frac{\partial L(\theta)}{\partial a_{i+1,1}} \\ \frac{\partial L(\theta)}{\partial a_{i+1,2}} \\ \vdots \\ \frac{\partial L(\theta)}{\partial a_{i+1,K}} \end{bmatrix}.$$

Also define the j -th column of the weight matrix:

$$W_{i+1,:j} = \begin{bmatrix} w_{i+1,1,j} \\ w_{i+1,2,j} \\ \vdots \\ w_{i+1,K,j} \end{bmatrix}.$$

Then,

$$\frac{\partial L(\theta)}{\partial h_{ij}} = (W_{i+1,:j})^T \nabla_{a_{i+1}} L(\theta).$$

Stacking the gradients for all hidden units gives

$$\nabla_{h_i} L(\theta) = \begin{bmatrix} \frac{\partial L(\theta)}{\partial h_{i1}} \\ \frac{\partial L(\theta)}{\partial h_{i2}} \\ \vdots \\ \frac{\partial L(\theta)}{\partial h_{in_i}} \end{bmatrix}.$$

Therefore, the hidden-layer gradient can be written compactly as

$$\boxed{\nabla_{h_i} L(\theta) = W_{i+1}^T \nabla_{a_{i+1}} L(\theta)}$$

which is one of the key backpropagation equations. It shows that the gradient at a hidden layer is obtained by propagating the gradient from the next layer through the transpose of the weight matrix.

2.2 Gradient with Respect to Hidden Activations

For hidden layers, we already know that

$$\nabla_{h_i} L(\theta) = W_{i+1}^T \nabla_{a_{i+1}} L(\theta).$$

However, during backpropagation we ultimately need the gradient with respect to the pre-activation vector a_i , because the parameters W_i and b_i directly affect a_i .

Recall that

$$h_{ij} = g(a_{ij}),$$

where $g(\cdot)$ is the activation function.

Applying the chain rule,

$$\frac{\partial L(\theta)}{\partial a_{ij}} = \frac{\partial L(\theta)}{\partial h_{ij}} \frac{\partial h_{ij}}{\partial a_{ij}} = \frac{\partial L(\theta)}{\partial h_{ij}} g'(a_{ij}).$$

Collecting all partial derivatives gives

$$\nabla_{a_i} L(\theta) = \begin{bmatrix} \frac{\partial L(\theta)}{\partial a_{i1}} \\ \frac{\partial L(\theta)}{\partial a_{i2}} \\ \vdots \\ \frac{\partial L(\theta)}{\partial a_{in_i}} \end{bmatrix} = \begin{bmatrix} \frac{\partial L(\theta)}{\partial h_{i1}} g'(a_{i1}) \\ \frac{\partial L(\theta)}{\partial h_{i2}} g'(a_{i2}) \\ \vdots \\ \frac{\partial L(\theta)}{\partial h_{in_i}} g'(a_{in_i}) \end{bmatrix}.$$

Using element-wise multiplication ($\odot$),

$$\boxed{\nabla_{a_i} L(\theta) = \nabla_{h_i} L(\theta) \odot g'(a_i)}$$

Substituting

$$\nabla_{h_i} L(\theta) = W_{i+1}^T \nabla_{a_{i+1}} L(\theta),$$

we obtain the backpropagation recursion

$$\boxed{\nabla_{a_i} L(\theta) = (W_{i+1}^T \nabla_{a_{i+1}} L(\theta)) \odot g'(a_i)}$$

which is valid for all hidden layers

$$i = L - 1, L - 2, \dots, 1.$$

2.3 Gradient with Respect to Weights

We now derive the gradient of the loss function with respect to the weight matrix W_k .

Recall that the pre-activation vector of layer k is

$$a_k = b_k + W_k h_{k-1}.$$

Consider a single element w_{kij} of W_k . By the chain rule,

$$\frac{\partial L(\theta)}{\partial w_{kij}} = \frac{\partial L(\theta)}{\partial a_{ki}} \frac{\partial a_{ki}}{\partial w_{kij}}.$$

For a layer with three hidden units,

$$\begin{bmatrix} a_{k1} \\ a_{k2} \\ a_{k3} \end{bmatrix} = \begin{bmatrix} b_{k1} \\ b_{k2} \\ b_{k3} \end{bmatrix} + \begin{bmatrix} w_{k11} & w_{k12} & w_{k13} \\ w_{k21} & w_{k22} & w_{k23} \\ w_{k31} & w_{k32} & w_{k33} \end{bmatrix} \begin{bmatrix} h_{k-1,1} \\ h_{k-1,2} \\ h_{k-1,3} \end{bmatrix}.$$

Hence,

$$a_{k1} = b_{k1} + w_{k11} h_{k-1,1} + w_{k12} h_{k-1,2} + w_{k13} h_{k-1,3}.$$

Therefore,

$$\frac{\partial a_{ki}}{\partial w_{kij}} = h_{k-1,j}.$$

Substituting into the chain rule,

$$\boxed{\frac{\partial L(\theta)}{\partial w_{kij}} = \frac{\partial L(\theta)}{\partial a_{ki}} h_{k-1,j}}$$

for every element of the weight matrix.

Stacking all partial derivatives together,

$$\nabla_{W_k} L(\theta) = \begin{bmatrix} \frac{\partial L(\theta)}{\partial w_{k11}} & \frac{\partial L(\theta)}{\partial w_{k12}} & \dots \\ \frac{\partial L(\theta)}{\partial w_{k21}} & \frac{\partial L(\theta)}{\partial w_{k22}} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

Using matrix notation, the gradient with respect to the weights becomes

$$\boxed{\nabla_{W_k} L(\theta) = \nabla_{a_k} L(\theta) h_{k-1}^T}$$

where

$$\nabla_{a_k} L(\theta) \in \mathbb{R}^{n_k}, \quad h_{k-1} \in \mathbb{R}^{n_{k-1}}.$$

The resulting matrix

$$\nabla_{W_k} L(\theta) \in \mathbb{R}^{n_k \times n_{k-1}}$$

has the same dimensions as W_k .

2.3.1 Example: Gradient Matrix for a 3×3 Weight Matrix

Consider a layer with

$$W_k \in \mathbb{R}^{3 \times 3}, \quad a_k \in \mathbb{R}^3, \quad h_{k-1} \in \mathbb{R}^3.$$

The weight matrix is

$$W_k = \begin{bmatrix} w_{k11} & w_{k12} & w_{k13} \\ w_{k21} & w_{k22} & w_{k23} \\ w_{k31} & w_{k32} & w_{k33} \end{bmatrix}.$$

The gradient of the loss with respect to the weight matrix is

$$\nabla_{W_k} L(\theta) = \begin{bmatrix} \frac{\partial L(\theta)}{\partial w_{k11}} & \frac{\partial L(\theta)}{\partial w_{k12}} & \frac{\partial L(\theta)}{\partial w_{k13}} \\ \frac{\partial L(\theta)}{\partial w_{k21}} & \frac{\partial L(\theta)}{\partial w_{k22}} & \frac{\partial L(\theta)}{\partial w_{k23}} \\ \frac{\partial L(\theta)}{\partial w_{k31}} & \frac{\partial L(\theta)}{\partial w_{k32}} & \frac{\partial L(\theta)}{\partial w_{k33}} \end{bmatrix}.$$

Using the chain rule,

$$\frac{\partial L(\theta)}{\partial w_{kij}} = \frac{\partial L(\theta)}{\partial a_{ki}} \frac{\partial a_{ki}}{\partial w_{kij}} = \frac{\partial L(\theta)}{\partial a_{ki}} h_{k-1,j}.$$

Therefore,

$$\nabla_{W_k} L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial a_{k1}} h_{k-1,1} & \frac{\partial L}{\partial a_{k1}} h_{k-1,2} & \frac{\partial L}{\partial a_{k1}} h_{k-1,3} \\ \frac{\partial L}{\partial a_{k2}} h_{k-1,1} & \frac{\partial L}{\partial a_{k2}} h_{k-1,2} & \frac{\partial L}{\partial a_{k2}} h_{k-1,3} \\ \frac{\partial L}{\partial a_{k3}} h_{k-1,1} & \frac{\partial L}{\partial a_{k3}} h_{k-1,2} & \frac{\partial L}{\partial a_{k3}} h_{k-1,3} \end{bmatrix}.$$

Define

$$\nabla_{a_k} L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial a_{k1}} \\ \frac{\partial L}{\partial a_{k2}} \\ \frac{\partial L}{\partial a_{k3}} \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$

and

$$h_{k-1}^T = \begin{bmatrix} h_{k-1,1} & h_{k-1,2} & h_{k-1,3} \end{bmatrix} \in \mathbb{R}^{1 \times 3}.$$

Then

$$\nabla_{a_k} L(\theta) h_{k-1}^T = \begin{bmatrix} \frac{\partial L}{\partial a_{k1}} \\ \frac{\partial L}{\partial a_{k2}} \\ \frac{\partial L}{\partial a_{k3}} \end{bmatrix} \begin{bmatrix} h_{k-1,1} & h_{k-1,2} & h_{k-1,3} \end{bmatrix}.$$

Expanding the outer product gives

$$= \begin{bmatrix} \frac{\partial L}{\partial a_{k1}} h_{k-1,1} & \frac{\partial L}{\partial a_{k1}} h_{k-1,2} & \frac{\partial L}{\partial a_{k1}} h_{k-1,3} \\ \frac{\partial L}{\partial a_{k2}} h_{k-1,1} & \frac{\partial L}{\partial a_{k2}} h_{k-1,2} & \frac{\partial L}{\partial a_{k2}} h_{k-1,3} \\ \frac{\partial L}{\partial a_{k3}} h_{k-1,1} & \frac{\partial L}{\partial a_{k3}} h_{k-1,2} & \frac{\partial L}{\partial a_{k3}} h_{k-1,3} \end{bmatrix}.$$

Hence,

$$\boxed{\nabla_{W_k} L(\theta) = \nabla_{a_k} L(\theta) h_{k-1}^T}$$

with dimensions

$$(3 \times 1)(1 \times 3) = 3 \times 3,$$

which matches the dimensions of W_k .

In general,

$$\nabla_{a_k} L(\theta) \in \mathbb{R}^{n_k \times 1}, \quad h_{k-1}^T \in \mathbb{R}^{1 \times n_{k-1}},$$

therefore

$$\boxed{\nabla_{W_k} L(\theta) = \nabla_{a_k} L(\theta) h_{k-1}^T \in \mathbb{R}^{n_k \times n_{k-1}}}$$

which is exactly the same shape as the weight matrix

$$W_k \in \mathbb{R}^{n_k \times n_{k-1}}.$$

2.4 Gradient with Respect to Biases

Recall that

$$a_{ki} = b_{ki} + \sum_j w_{kij} h_{k-1,j}.$$

Applying the chain rule,

$$\frac{\partial L(\theta)}{\partial b_{ki}} = \frac{\partial L(\theta)}{\partial a_{ki}} \frac{\partial a_{ki}}{\partial b_{ki}}.$$

Since

$$\frac{\partial a_{ki}}{\partial b_{ki}} = 1,$$

we obtain

$$\boxed{\frac{\partial L(\theta)}{\partial b_{ki}} = \frac{\partial L(\theta)}{\partial a_{ki}}}$$

for every bias term.

Therefore,

$$\nabla_{b_k} L(\theta) = \begin{bmatrix} \frac{\partial L(\theta)}{\partial b_{k1}} \\ \frac{\partial L(\theta)}{\partial b_{k2}} \\ \vdots \\ \frac{\partial L(\theta)}{\partial b_{kn_k}} \end{bmatrix} = \begin{bmatrix} \frac{\partial L(\theta)}{\partial a_{k1}} \\ \frac{\partial L(\theta)}{\partial a_{k2}} \\ \vdots \\ \frac{\partial L(\theta)}{\partial a_{kn_k}} \end{bmatrix}.$$

Hence,

$$\boxed{\nabla_{b_k} L(\theta) = \nabla_{a_k} L(\theta)}$$

which completes the derivation of gradients with respect to hidden activations, weights, and biases used in the backpropagation algorithm.

2.5 Gradient Descent Algorithm

We have already derived

$$\nabla_{a_k} L(\theta) \quad (\text{gradient w.r.t. output layer})$$

$$\nabla_{h_k} L(\theta), \quad \nabla_{a_k} L(\theta) \quad \text{for hidden layers } (0 < k < L)$$

and

$$\nabla_{W_k} L(\theta), \quad \nabla_{b_k} L(\theta) \quad (\text{gradients w.r.t. weights and biases}).$$

We are now ready to describe the complete gradient descent algorithm.

2.6 Algorithm: gradient-descent()

$$t \leftarrow 0$$

Initialize,

$$\theta_0 = [W_1^0, \dots, W_K^0, b_1^0, \dots, b_K^0]$$

while $t + 1 < \text{max-iteration}$ do

$$h_1, h_2, \dots, h_{K-1}, a_1, a_2, \dots, a_K, \hat{y} = \text{forward-propagation}(\theta_t),$$

$$\nabla_{\theta_t} L = \text{back-propagation}(h_1, h_2, \dots, h_{K-1}, a_1, a_2, \dots, a_K, \hat{y}),$$

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla_{\theta_t} L$$

end.

2.7 Algorithm: forward-propagation(θ)

for $k = 1$ to $L - 1$ do

$$a_k = b_k + W_k h_{k-1}$$

$$h_k = g(a_k)$$

end

$$a_L = b_L + W_L h_{L-1}$$

$$\hat{y} = O(a_L).$$

2.8 Algorithm: back-propagation

// Compute output gradient

$$\nabla_{a_L} L(\theta) = -(e(\hat{y}) - f(x))$$

for $k = L$ to 1 do

// Compute gradients w.r.t. parameter

$$\nabla_{W_k} L(\theta) = \nabla_{a_k} L(\theta) h_{k-1}^T$$

$$\nabla_{b_k} L(\theta) = \nabla_{a_k} L(\theta)$$

// Compute gradient w.r.t. layer below

$$\nabla_{h_{k-1}} L(\theta) = W_k^T (\nabla_{a_k} L(\theta))$$

// Compute gradients w.r.t. layer below (pre-active)

$$\nabla_{a_{k-1}} L(\theta) = \nabla_{h_{k-1}} L(\theta) \cdot \left[\dots, g'(a_{k-1,j}), \dots \right]$$

end.

3 UNIT 2

3.1 Deep Gradient Descent

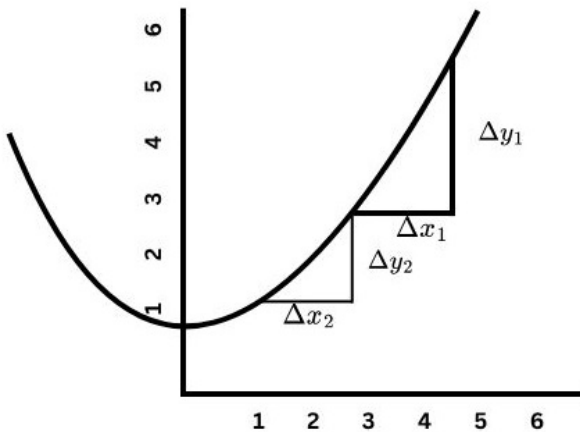


Figure 8: Gradient Descent Recap

The larger the gradient,

$$\frac{\Delta y_i}{\Delta x_i},$$

the larger the update.

- When the error is large, the gradient

$$\frac{\Delta y_i}{\Delta x_i}$$

is large.

- Hence, larger weight updates are made,

$$\Delta w = \eta \nabla_w L.$$

- Thus, in regions where the error surface is steep, the parameter updates are large.

Irrespective of where we start, once we reach a region of the error surface having a gentle slope, the progress slows down.

The implication of the contour maps in gradient descent is that the gradient indicates how steep or how gentle the error surface is.

- If the surface is steep, the gradient magnitude is large, resulting in larger parameter updates.
- If the surface is flat or gently sloped, the gradient magnitude becomes small, resulting in smaller updates.
- Consequently, gradient descent naturally takes large steps on steep regions and small steps near the optimum.

This adaptive behaviour allows gradient descent to move rapidly towards regions of lower error while avoiding excessively large updates near the minimum of the loss function.

3.2 Momentum-Based Gradient Descent

Intuition

- If I am repeatedly being asked to move in the same direction, then I should probably gain some confidence and start taking bigger steps in that direction.
- Just as a ball gains momentum while rolling down a slope.

Update Rule for Momentum-Based Gradient Descent

$$\begin{aligned} \text{update}_t &= \gamma \text{update}_{t-1} + \eta \nabla W_t, \\ W_{t+1} &= W_t - \text{update}_t. \end{aligned}$$

where

- γ is the momentum coefficient ($0 < \gamma < 1$),
- η is the learning rate,
- ∇W_t is the gradient at iteration t .

Let us break down the update rule

Initially,

$$\text{update}_0 = 0.$$

Therefore,

$$\text{update}_1 = \gamma \text{update}_0 + \eta \nabla W_1 = \eta \nabla W_1.$$

Next,

$$\begin{aligned} \text{update}_2 &= \gamma \text{update}_1 + \eta \nabla W_2 \\ &= \gamma \eta \nabla W_1 + \eta \nabla W_2. \end{aligned}$$

We are not only following the previous direction $\eta \nabla W_{t-1}$, but also considering the entire history of gradients.

Hence,

$$\begin{aligned} \text{update}_3 &= \gamma \text{update}_2 + \eta \nabla W_3 \\ &= \gamma (\gamma \eta \nabla W_1 + \eta \nabla W_2) + \eta \nabla W_3 \\ &= \gamma^2 \eta \nabla W_1 + \gamma \eta \nabla W_2 + \eta \nabla W_3. \end{aligned}$$

Similarly,

$$\begin{aligned} \text{update}_4 &= \gamma \text{update}_3 + \eta \nabla W_4 \\ &= \gamma^3 \eta \nabla W_1 + \gamma^2 \eta \nabla W_2 + \gamma \eta \nabla W_3 + \eta \nabla W_4. \end{aligned}$$

This is an **Exponentially Weighted Average (EWA)** of all previous gradients.

In general,

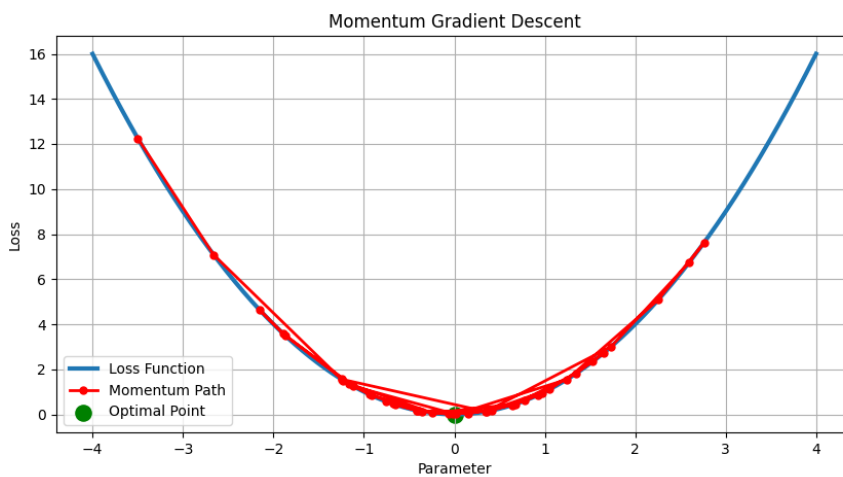
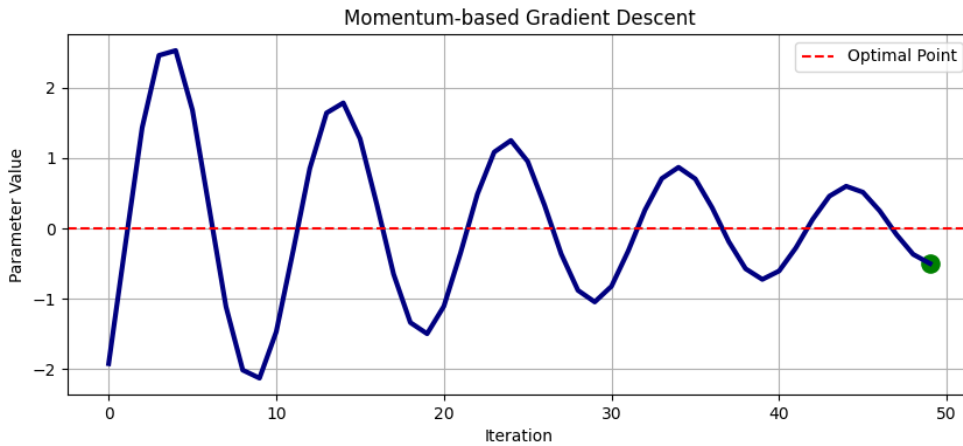
$$\text{update}_t = \gamma^{t-1} \eta \nabla W_1 + \gamma^{t-2} \eta \nabla W_2 + \cdots + \gamma \eta \nabla W_{t-1} + \eta \nabla W_t.$$

Notice that older gradients are multiplied by higher powers of γ . Hence, we gradually place less confidence on very old gradients while retaining useful information from recent updates.

Observation

Even in regions having gentle slopes, momentum-based gradient descent is able to take larger steps because it accumulates the history of gradients instead of relying only on the current gradient.

3.2.1 Limitations of Momentum-Based Gradient Descent



- In moving fast, there is always a possibility that momentum carries the optimization beyond the optimum point.
- Let us change the input data so that we obtain a different error surface.
- Momentum-based gradient descent oscillates in and out of the minimum valley because the accumulated momentum carries the optimization across the valley.
- As a result, it may require several oscillations before finally converging to the optimum.
- Despite these oscillations, momentum-based gradient descent usually converges faster than vanilla gradient descent.

- Consider vanilla gradient descent. Suppose the error is still relatively high, but the slope has become very gentle. Since the gradient magnitude is very small, it becomes difficult to determine whether the algorithm has actually reached the optimum or is simply passing through a flat region.
- In contrast, for momentum-based gradient descent, when the optimization finally comes to a stop, we can confidently conclude that the algorithm is very close to the point of convergence. Hence, the optimization process can be terminated.

3.3 Nesterov Accelerated Gradient Descent (NAG)

3.3.1 Motivation

- The problem with momentum-based gradient descent is that it oscillates significantly before reaching the stationary point.
- The main idea behind Nesterov Accelerated Gradient is:

“Look before you leap.”

- Recall the update rule for momentum-based gradient descent:

$$\begin{aligned}\text{update}_t &= \gamma \text{update}_{t-1} + \eta \nabla W_t, \\ W_{t+1} &= W_t - \text{update}_t.\end{aligned}$$

This can be viewed as a two-step process:

$$W_t \xrightarrow{\gamma \text{update}_{t-1}} \text{Intermediate Point} \xrightarrow{\eta \nabla W_t} W_{t+1}.$$

We already know that we are going to move by at least

$$\gamma \text{update}_{t-1},$$

and then move a little more according to

$$\eta \nabla W_t.$$

Therefore, instead of computing the gradient at the current parameter value, why not compute the gradient after taking the momentum step?

Define the **look-ahead point** as

$$W_{\text{look-ahead}} = W_t - \gamma \text{update}_{t-1}.$$

Instead of evaluating

$$\nabla W_t,$$

we evaluate

$$\nabla W_{\text{look-ahead}}.$$

Hence, the update rule becomes

$$W_{\text{look-ahead}} = W_t - \gamma \text{update}_{t-1}$$

$$\text{update}_t = \gamma \text{update}_{t-1} + \eta \nabla W_{\text{look-ahead}}$$

$$W_{t+1} = W_t - \text{update}_t$$

The same update rule is repeated for every iteration.

Since the optimizer already possesses momentum from previous iterations, it carries the optimization in the expected direction.

Rather than computing the gradient at the current point, NAG computes the gradient at the look-ahead point.

Consequently,

- the oscillations are reduced,
- the optimization becomes more stable,
- convergence is achieved more quickly.

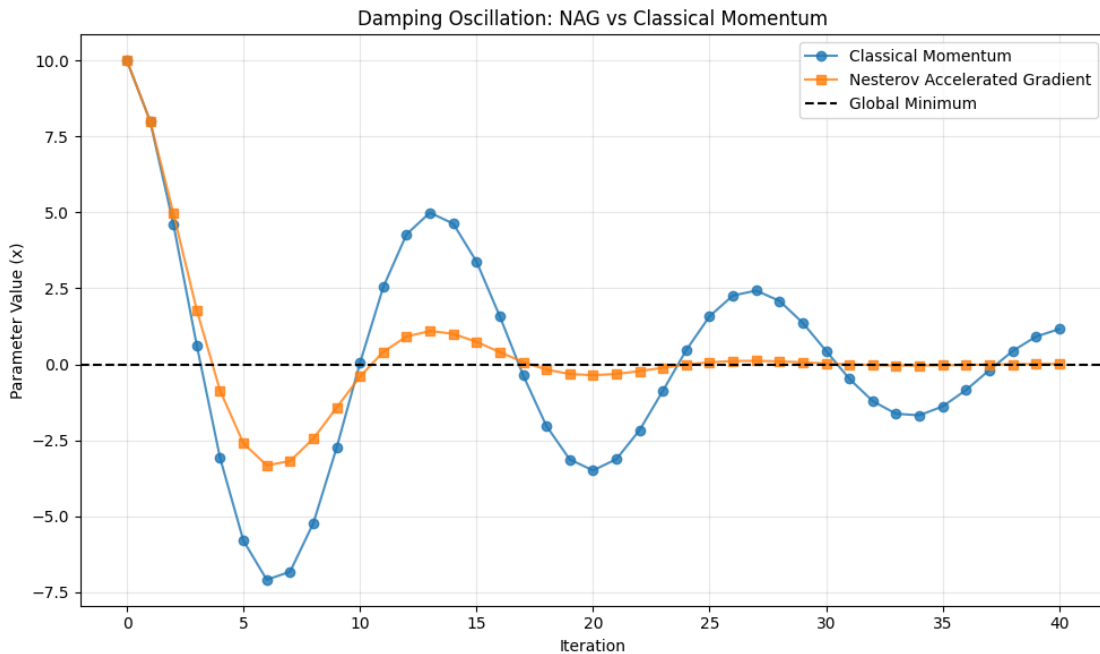


Figure 9: NAG Vs. Momentum Based GD

3.4 Comparison of Classical Momentum and Nesterov Accelerated Gradient (NAG)

Figure 9 compares the optimization behavior of **Classical Momentum Gradient Descent** and **Nesterov Accelerated Gradient (NAG)**. The horizontal axis represents the number of iterations, while the vertical axis represents the parameter value being optimized. The dashed horizontal line at $y = 0$ indicates the global optimum.

Initially, both optimization algorithms start from the same parameter value and move rapidly toward the optimum because the gradient magnitude is large. During the first few iterations, both methods exhibit nearly identical trajectories as they accelerate toward the minimum.

The blue curve represents **Classical Momentum Gradient Descent**. As the optimizer approaches the optimum, the accumulated momentum causes the parameter to overshoot the minimum. Consequently, the parameter oscillates around the optimum with a relatively large amplitude before gradually converging.

The orange curve represents **Nesterov Accelerated Gradient (NAG)**. Unlike Classical Momentum, NAG computes the gradient at a predicted future position rather than the current position. This look-ahead strategy enables the optimizer to anticipate overshooting and apply corrective updates before crossing the optimum.

The graph clearly illustrates that the oscillations produced by NAG are significantly smaller than those of Classical Momentum. As a result, the oscillations decay more rapidly, allowing the parameter to stabilize near the optimum in fewer iterations.

Overall, Nesterov Accelerated Gradient demonstrates faster and smoother convergence compared to Classical Momentum Gradient Descent. Its predictive gradient computation reduces overshooting, minimizes oscillations, and improves optimization efficiency, making it a preferred choice for training deep neural networks.

3.4.1 Observations

- By looking ahead, Nesterov Accelerated Gradient (NAG) corrects its course earlier than momentum-based gradient descent.
- As a result, oscillations become smaller.
- The optimizer has a higher chance of escaping shallow local minima.
- NAG generally converges faster than classical momentum-based gradient descent.

4 Stochastic and Mini-Batch Gradient Descent

- The GD algorithm goes over the entire data once before updating the parameters.
- Why? Because this is the true gradient of the loss as derived earlier (sum of the gradients of the losses corresponding to each data point).

- No approximation. Hence, theoretical guarantees hold (in other words, each step guarantees that the loss will decrease).
- What's the flip side? Imagine we have a million training data points. To make one update to w, b , the algorithm makes a million calculations. So obviously, it is very slow.
- Can we do better?

4.1 Vanilla Gradient Descent

Algorithm

For all points in the dataset (X, Y) do

$$\Delta W = \Delta W + \nabla_W \mathcal{L}(W, b, x_i, y_i)$$

$$\Delta b = \Delta b + \nabla_b \mathcal{L}(W, b, x_i, y_i)$$

end

Update the parameters:

$$W = W - \eta \Delta W$$

$$b = b - \eta \Delta b$$

Observation

- It is too slow, as it may have to go through millions of data points before performing one update.
- What direction do we have here?
- Instead of updating after processing all the data points, we can update based on a single data point so that we can speed up the process.

Modification

For every data point (x_i, y_i) in the training set, do

$$\Delta W = \Delta W + \nabla_W \mathcal{L}(W, b, x_i, y_i)$$

$$\Delta b = \Delta b + \nabla_b \mathcal{L}(W, b, x_i, y_i)$$

$$W = W - \eta \Delta W$$

$$b = b - \eta \Delta b$$

end

4.2 Observation

- What is the flip side?

We need to randomly choose each point and update the parameters without considering the historical updates. Hence, it may lead to errors.

- Practically, we should avoid population decisions and instead make gradient updates based on sampled data.
- Now we are training on a sample point, and we are claiming that the whole population is following the same probability as the sampled point.
- There is no guarantee that each update step will decrease the loss.



Figure 10: Stochastic GD

Why?

- Because we are making greedy decisions.
- Each data point tries to push the parameters in the direction that is most favorable to itself, without being aware of how this affects the remaining data points.
- Consequently, the optimization may oscillate significantly, and successive updates may move in conflicting directions.

4.3 Mini-Batch Gradient Descent

- A parameter update that is locally favorable for one point may harm other points. It is almost as if the data points are competing with each other.
- Can we reduce this oscillation?

We can do it with **Mini-Batch Gradient Descent (Mini-Batch GD)**.

- What arrangements do we need to make?

For each point in the dataset of the form (X, Y) ,

$$\Delta W = \Delta W + \nabla_W \mathcal{L}(W, b, X, Y)$$

$$\Delta b = \Delta b + \nabla_b \mathcal{L}(W, b, X, Y)$$

No. of processed points $\leftarrow$ No. of processed points + 1

If the number of processed points reaches the mini-batch size,

$$W \leftarrow W - \eta \Delta W$$

$$b \leftarrow b - \eta \Delta b$$

- Notice that the algorithm updates the parameters only after it has seen *mini-batch-size* number of data points.
- Therefore, the stochastic estimates are now slightly better.

4.4 Mini-Batch Gradient Descent: Some Important Facts

- If we take $B = 2$, it will be slightly better.
- Analogy: we are now tossing the coin less frequently.
- The higher the value of B , the smaller the oscillations.
- Typically, we choose

$$B \in \{16, 32, 64\}.$$

- Of course, some oscillation will still be present.

Some Important Facts

1 Epoch = One complete pass over the entire dataset

1 Step = One parameter update

N = Number of data points

B = Mini-batch size

Algorithm	# Steps in One Epoch
Vanilla Gradient Descent	1
Stochastic Gradient Descent	N
Mini-Batch Gradient Descent	$\frac{N}{B}$

4.5 Gradient Descent with Adaptive Learning Rate

Motivation

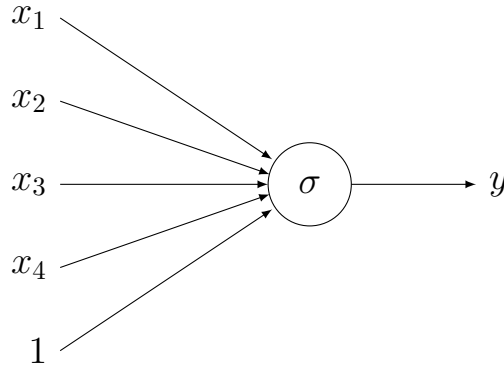


Figure 11: Single Neuron with Sigmoid Activation

The output of the neuron is

$$y = f(x) = \frac{1}{1 + e^{-(W^T X + b)}}.$$

where

$$X = \{x_1, x_2, x_3, x_4\},$$

and

$$W = \{w_1, w_2, w_3, w_4\}.$$

- Given this network, for a single training example (X, y) , the gradient corresponding to each weight can be computed independently.

For example,

$$\nabla_{w_1} \mathcal{L} = (\hat{y} - y) \hat{y}(1 - \hat{y}) x_1,$$

$$\nabla_{w_2} \mathcal{L} = (\hat{y} - y) \hat{y}(1 - \hat{y}) x_2,$$

and similarly for the remaining weights.

- If there are n training samples, we simply sum the gradients over all samples to obtain the total gradient.

- What happens if one feature is very sparse?

Suppose we have one million training samples. Although gradients are computed over the entire dataset, a sparse feature may appear only a few times. Consequently, the corresponding parameter receives very few updates.

- Therefore, if a feature is sparse, its associated weight is not updated frequently.
- If such a sparse feature is also important, we should update its corresponding weight more aggressively.
- This motivates the following question:

Can we assign a different learning rate to each parameter so that infrequently updated parameters receive larger updates?

4.6 AdaGrad (Adaptive Gradient)

Intuition

- Decay the learning rate for each parameter in proportion to its update history.

Update Rule for AdaGrad

Accumulate the squared gradients:

$$v_t = v_{t-1} + (\nabla W_t)^2,$$

where the square is taken element-wise. The direction of the gradient is not important here; only its magnitude matters.

The parameter update becomes

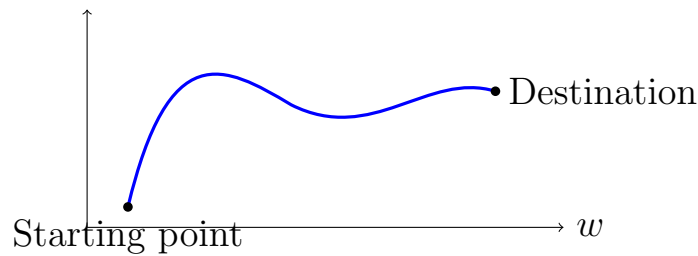
$$W_{t+1} = W_t - \frac{\eta}{\sqrt{v_t + \varepsilon}} \odot \nabla W_t,$$

Similarly, for the bias,

$$b_{t+1} = b_t - \frac{\eta}{\sqrt{v_t + \varepsilon}} \odot \nabla b_t.$$

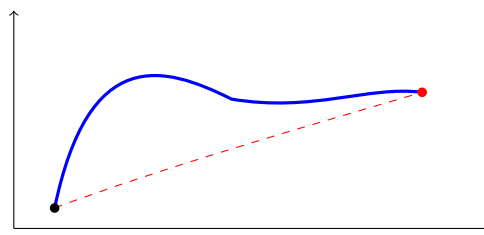
- We divide the learning rate by the accumulated history.
- For dense features, the accumulated gradient is large. Therefore, the effective learning rate becomes smaller.
- For sparse features, the accumulated gradient is small. Hence, the effective learning rate remains relatively larger.

Suppose a dataset contains nearly 80% sparse features. Most feature weights receive only a few updates. In such cases, algorithms such as Gradient Descent, Momentum, and NAG may exhibit slow progress.



Observation

- This behavior occurs because the feature corresponding to parameter W is sparse. Consequently, W undergoes very few updates.
- On the other hand, the bias parameter b is dense and therefore receives many updates.
- As a result, there is much more movement along the b direction than along the W direction.
- AdaGrad addresses this problem by assigning parameter-specific learning rates.



- By using parameter-specific learning rates, AdaGrad ensures that sparse parameters receive larger updates because they have smaller accumulated gradient histories.
- Conversely, parameters that receive frequent updates experience a reduced effective learning rate because the denominator $\sqrt{v_t + \varepsilon}$ keeps increasing.
- In practice, AdaGrad often decreases the learning rate too aggressively. To avoid division by zero and improve numerical stability, a small constant ε is added inside the square root.

4.7 Limitation of AdaGrad

Problem

- One drawback of AdaGrad is that the effective learning rate continuously decreases over time.
- Eventually, the learning rate becomes extremely small, and further parameter updates become negligible.

Motivation

- AdaGrad decays the learning rate very aggressively.
- As training progresses, frequently updated parameters receive extremely small updates because their effective learning rate has decayed significantly.
- To avoid this issue, instead of allowing the denominator to grow indefinitely, we control its growth using an exponential moving average.

4.8 Update Rule for RMSProp

Instead of accumulating all past squared gradients,

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla W_t)^2,$$

where β is the decay factor.

The parameter update becomes

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{v_t + \varepsilon}} \nabla W_t.$$

Similarly, for the bias parameter,

$$b_{t+1} = b_t - \frac{\eta}{\sqrt{v_t + \varepsilon}} \nabla b_t.$$

A commonly used value is

$$\beta = 0.95.$$

Example

Assume

$$v_0 = 0.$$

Then,

$$v_1 = (1 - \beta)(\nabla W_1)^2 = 0.05(\nabla W_1)^2.$$

The next update becomes

$$v_2 = 0.95 \times 0.05(\nabla W_1)^2 + 0.05(\nabla W_2)^2.$$

Thus, instead of accumulating all previous gradients equally, RMSProp maintains an exponentially weighted moving average of squared gradients.

Observations

- Unlike AdaGrad, RMSProp does not allow the accumulated gradient history to grow without bound.
- The effective learning rate decreases in a controlled manner rather than aggressively.
- AdaGrad often becomes almost stationary near convergence because the learning rate becomes too small.
- RMSProp prevents this behavior and allows the optimizer to continue making meaningful updates, especially along directions where AdaGrad would almost stop learning.
- Consequently, RMSProp generally converges faster and more reliably than AdaGrad for deep neural networks.

4.9 Adam (Adaptive Moment Estimation)

Motivation

- Adam combines the advantages of both Momentum Gradient Descent and RMSProp.
- Momentum keeps an exponentially weighted average of the gradients (first moment).
- RMSProp keeps an exponentially weighted average of the squared gradients (second moment).
- Typical values are

$$\beta_1 = 0.9, \quad \beta_2 = 0.999.$$

Update Rules

The first moment estimate is

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla W_t.$$

The second moment estimate is

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla W_t)^2.$$

Since both estimates are initialized at zero, bias correction is applied:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t},$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Finally, the parameter update is

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t.$$

4.10 Why Bias Correction is Needed

Assume that the stochastic gradient is a random variable

$$g_t = \nabla W_t.$$

We compute the exponentially weighted moving average

$$m_t = \beta m_{t-1} + (1 - \beta) g_t,$$

with

$$m_0 = 0.$$

The first few iterations are

$$m_1 = (1 - \beta) g_1,$$

$$m_2 = \beta(1 - \beta) g_1 + (1 - \beta) g_2,$$

$$m_3 = \beta^2(1 - \beta) g_1 + \beta(1 - \beta) g_2 + (1 - \beta) g_3.$$

In general,

$$m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i.$$

Taking expectation,

$$\mathbb{E}[m_t] = (1 - \beta) \sum_{i=1}^t \beta^{t-i} \mathbb{E}[g_i].$$

Assuming all gradients have the same expectation,

$$\mathbb{E}[g_i] = \mathbb{E}[g],$$

we obtain

$$\mathbb{E}[m_t] = \mathbb{E}[g] (1 - \beta) \sum_{i=1}^t \beta^{t-i}.$$

Using the geometric series,

$$\sum_{i=1}^t \beta^{t-i} = \frac{1 - \beta^t}{1 - \beta},$$

therefore,

$$\mathbb{E}[m_t] = (1 - \beta^t) \mathbb{E}[g]$$

which shows that m_t is a biased estimate of the true mean. Hence,

$$\hat{m}_t = \frac{m_t}{1 - \beta^t}$$

is used as the unbiased first-moment estimate. Similarly,

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

is the unbiased estimate of the second moment.

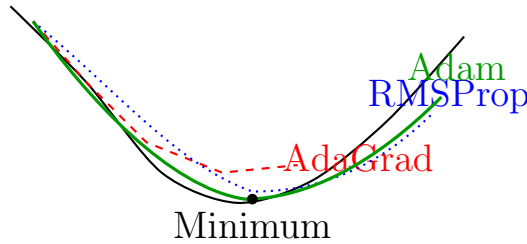


Figure 12: Illustration of optimization paths using Gradient Descent, RMSProp, and Adam.

Comparison of Gradient Descent, RMSProp, and Adam

Figure 12 illustrates the optimization trajectories of three popular optimization algorithms. Vanilla Gradient Descent (GD) follows a fixed learning rate throughout training, which often results in slow convergence and oscillations, particularly in regions where the loss surface has steep curvature. Consequently, GD typically requires more iterations to reach the optimum.

RMSProp improves upon Gradient Descent by assigning an adaptive learning rate to each parameter based on the exponentially weighted average of its past squared gradients. Parameters that receive frequent updates are assigned a smaller effective learning rate, while infrequently updated (sparse) parameters retain a relatively larger learning rate. This enables RMSProp to converge faster and with reduced oscillation.

Adam (Adaptive Moment Estimation) combines the strengths of Momentum and RMSProp by simultaneously maintaining estimates of the first moment (mean of gradients) and the second moment (uncentered variance of gradients). The momentum term accelerates optimization in the correct direction, whereas the adaptive learning

rate stabilizes the updates. As a result, Adam generally converges faster, follows a smoother optimization path, and exhibits significantly less oscillation than both Gradient Descent and RMSProp. These characteristics make Adam one of the most widely used optimization algorithms for training deep neural networks.

Table 3: Comparison of learning-rate optimization algorithms.

Optimizer	Learning Rate	Convergence	Characteristics
Gradient Descent	Fixed	Slow	Prone to oscillation, requires careful tuning of the global learning rate.
RMSProp	Adaptive (per parameter)	Moderate to Fast	Reduces oscillation by scaling the learning rate using historical squared gradients. Performs well for sparse features.
Adam	Adaptive with Momentum	Fast	Combines momentum and adaptive learning rates, resulting in faster convergence, smoother optimization, and improved stability.

5 Linear Algebra: Eigenvalues and Eigenvectors

This chapter introduces the fundamental concepts of linear algebra that are essential for understanding machine learning and deep learning algorithms. The major topics covered are:

1. Linear equations
2. Matrix algebra
3. Vector spaces and basis
4. Orthogonality
5. Eigenvalues and Eigenvectors

5.1 Linear Equation

A **linear equation** is an algebraic equation in which every term is either:

1. A constant, or
2. The product of a constant and a variable.

Examples of linear equations are

$$3x + 2y = 7, \quad 5x - y = 10.$$

Example Consider two vectors

$$\mathbf{u} = 4\mathbf{i}, \quad \mathbf{v} = 3\mathbf{i}.$$

Since both vectors are scalar multiples of the same basis vector, they lie on the same line. More generally, any linear combination

$$a\mathbf{u} + b\mathbf{v}$$

lies in the span of the vectors.

5.2 System of Linear Equations

A system of linear equations consists of m linear equations with n unknown variables.

The general form is

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m. \end{aligned}$$

Depending on the values of m and n ,

- If $m < n$, the system is underdetermined.
- If $m = n$, the coefficient matrix is square.
- If $m > n$, the system is overdetermined.

One of the most popular techniques for solving a system of linear equations is the **Gaussian Elimination** method.

5.3 What is a Matrix?

Before solving systems of equations, it is important to understand matrices.

According to **Gilbert Strang (MIT)**,

”A matrix is a way to describe a linear transformation.”

According to **Carl D. Meyer**,

”A matrix is a mathematical device for recording and working with linear equations and linear transformations.”

5.4 Example: Matrix Representation

Consider the following system of equations:

$$3x + 5y + 6z = 20,$$

$$4x + 3y + 2z = 10,$$

$$5x + 2y + z = 11.$$

This system can be written in matrix form as

$$\underbrace{\begin{bmatrix} 3 & 5 & 6 \\ 4 & 3 & 2 \\ 5 & 2 & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x \\ y \\ z \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 20 \\ 10 \\ 11 \end{bmatrix}}_{\mathbf{b}}.$$

Hence,

$$\boxed{A\mathbf{x} = \mathbf{b}}$$

where

- A is the coefficient matrix,
- $\mathbf{x}$ is the vector of unknown variables,
- $\mathbf{b}$ is the constant vector.

6 Inverse Matrix

The concept of matrices has a rich historical background.

1. Around **200 B.C.**, Chinese mathematicians, in the book *The Nine Chapters on the Mathematical Art*, used matrix-like methods to solve systems of linear equations.
2. During the **18th and 19th centuries**, matrix theory was developed further.

3. In **1858**, **Arthur Cayley** established the foundations of modern matrix theory and published the first formal definition of a matrix in his famous paper,

"A Memoir on the Theory of Matrices."

Several mathematicians made significant contributions to matrix theory, including

1. James Joseph Sylvester
2. Gottfried Wilhelm Leibniz
3. Carl Friedrich Gauss
4. Arthur Cayley
5. Ferdinand Georg Frobenius

6.1 Inverse Matrix using Gaussian Elimination

Consider the linear system

$$A\mathbf{x} = \mathbf{b}.$$

If the inverse of A exists, then

$$A^{-1}A\mathbf{x} = A^{-1}\mathbf{b},$$

which gives

$$\boxed{\mathbf{x} = A^{-1}\mathbf{b}}.$$

The inverse of a matrix can be obtained using Gaussian elimination by augmenting the coefficient matrix with the identity matrix,

$$[A \mid I] \longrightarrow [I \mid A^{-1}].$$

This transformation is possible only when A is invertible.

Condition for Existence of Inverse A square matrix has an inverse only if

$$\boxed{\det(A) \neq 0}.$$

If

$$\det(A) = 0,$$

then the matrix is called **singular** and its inverse does not exist.

7 Matrix Algebra

7.1 Matrix Addition

Two matrices can be added only if they have the same dimensions.

For example,

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}.$$

For two matrices $A = [a_{ij}]$ and $B = [b_{ij}]$,

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

7.2 Matrix Subtraction

Similarly,

$$(A - B)_{ij} = a_{ij} - b_{ij}.$$

7.3 Properties of Matrix Addition

Let A , B , and C be matrices of size $m \times n$. Then the following properties hold.

1. Closure Property

$$A + B$$

is also an $m \times n$ matrix.

2. Associative Property

$$(A + B) + C = A + (B + C).$$

3. Commutative Property

$$A + B = B + A.$$

4. Additive Identity

There exists a zero matrix O such that

$$A + O = A.$$

5. Additive Inverse

Every matrix A has an additive inverse $-A$ satisfying

$$A + (-A) = O.$$

8 Scalar Multiplication

If $A = [a_{ij}]$ is an $m \times n$ matrix and α is a scalar, then scalar multiplication is defined as

$$\alpha A = [\alpha a_{ij}].$$

Example

$$2 \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 8 & 10 \end{bmatrix}.$$

9 Transpose of a Matrix

The **transpose** of an $m \times n$ matrix A is obtained by interchanging its rows and columns. The transpose is denoted by A^T .

If

$$A = [a_{ij}],$$

then

$$(A^T)_{ij} = a_{ji}.$$

Thus, an $m \times n$ matrix becomes an $n \times m$ matrix after transposition.

Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix},$$

then

$$A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}.$$

10 Conjugate Matrix

For a complex matrix

$$A = [a_{ij}],$$

its **complex conjugate** is denoted by

$$\overline{A} = [\overline{a_{ij}}],$$

where every complex entry is replaced by its complex conjugate.

Example

$$A = \begin{bmatrix} 1 - 4i & i & 2 \\ 3 & 2 + i & 0 \end{bmatrix},$$

then

$$\overline{A} = \begin{bmatrix} 1 + 4i & -i & 2 \\ 3 & 2 - i & 0 \end{bmatrix}.$$

11 Conjugate Transpose (Hermitian Transpose)

The **conjugate transpose** of a matrix is obtained by first taking the complex conjugate and then taking the transpose.

It is denoted by

$$A^H = (\overline{A})^T.$$

Equivalently,

$$(A^H)_{ij} = \overline{a_{ji}}.$$

Example Using the previous matrix,

$$A^H = \begin{bmatrix} 1 + 4i & 3 \\ -i & 2 - i \\ 2 & 0 \end{bmatrix}.$$

12 Properties of the Transpose

For matrices A , B of the same order and scalar α ,

1. Transpose of a Sum

$$(A + B)^T = A^T + B^T.$$

2. Transpose of Scalar Multiplication

$$(\alpha A)^T = \alpha A^T.$$

3. Conjugate of a Sum

$$\overline{(A + B)} = \overline{A} + \overline{B}.$$

4. Conjugate of Scalar Multiplication

$$\overline{\alpha A} = \overline{\alpha} \overline{A}.$$

13 Important Types of Matrices

1. Symmetric Matrix

A square matrix A is called symmetric if

$$A = A^T.$$

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}.$$

2. Skew-Symmetric Matrix

A square matrix A is called skew-symmetric if

$$A = -A^T.$$

Example:

$$\begin{bmatrix} 0 & 2 & -1 \\ -2 & 0 & 4 \\ 1 & -4 & 0 \end{bmatrix}.$$

14 Example: Basis in $\mathbb{R}^2$

Consider the vectors

$$v_1 = (1, 0), \quad v_2 = (0, 1).$$

Any vector $(x, y) \in \mathbb{R}^2$ can be expressed as

$$(x, y) = xv_1 + yv_2.$$

Hence, the vectors $\{v_1, v_2\}$ form a basis of $\mathbb{R}^2$.

However, if

$$v_1 = (1, 0), \quad v_2 = (2, 0),$$

then

$$v_2 = 2v_1,$$

which means the vectors are linearly dependent and therefore cannot form a basis.

15 Spanning Set

Let

$$S = \{v_1, v_2, \dots, v_q\}$$

be a set of vectors.

The collection of all possible linear combinations of vectors in S is called the **span** of S and is defined as

$$\text{Span}(S) = \{\alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_q v_q : \alpha_i \in \mathbb{R}\}.$$

The span of S is the subspace generated by the vectors in S .

If V is a vector space, then

$$V = \text{Span}(S)$$

means that every vector in V can be written as a linear combination of vectors from S .

16 Basis

A set of vectors

$$B = \{v_1, v_2, \dots, v_n\}$$

is called a **basis** of a vector space V if

1. the vectors are linearly independent, and
2. they span the entire vector space V .

Thus,

$$V = \text{Span}(B).$$

17 Vector Space

A **vector space** is a non-empty set of vectors together with two algebraic operations:

1. Vector Addition

$$\mathbf{x} + \mathbf{y}$$

2. Scalar Multiplication

$$\alpha \mathbf{x},$$

where α is a scalar.

These operations satisfy the vector space axioms.

18 Linear Combination

Suppose

$$v_1, v_2, \dots, v_n$$

are vectors.

A vector of the form

$$a_1v_1 + a_2v_2 + \dots + a_nv_n,$$

where

$$a_1, a_2, \dots, a_n$$

are scalars, is called a **linear combination** of the vectors.

19 Linearly Independent Vectors

The vectors

$$v_1, v_2, \dots, v_n$$

are said to be **linearly independent** if

$$a_1v_1 + a_2v_2 + \dots + a_nv_n = \mathbf{0}$$

implies

$$a_1 = a_2 = \dots = a_n = 0.$$

In other words, the only way to obtain the zero vector as a linear combination is by choosing all coefficients equal to zero.

Consequently, none of the vectors can be represented as a linear combination of the remaining vectors.

20 Linearly Dependent Vectors

The vectors

$$v_1, v_2, \dots, v_n$$

are said to be **linearly dependent** if there exist scalars, not all zero, such that

$$a_1v_1 + a_2v_2 + \dots + a_nv_n = \mathbf{0}.$$

In this case, at least one vector can be expressed as a linear combination of the others.

21 Relationship Between Span and Basis

- A spanning set generates the entire vector space.
- A basis is a spanning set whose vectors are also linearly independent.
- Every vector in the space has a unique representation as a linear combination of basis vectors.

22 Dimension of a Vector Space

The **dimension** of a vector space V is defined as the number of vectors in any basis of V .

$$\dim(V) = \text{Number of vectors in a basis of } V$$

For example,

$$\dim(\mathbb{R}^2) = 2, \quad \dim(\mathbb{R}^3) = 3.$$

23 Determinant

The **determinant** is a scalar value associated with a square matrix.

For a matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

its determinant is

$$\det(A) = |A| = ad - bc.$$

23.1 Geometric Meaning of Determinant

The determinant measures how much a linear transformation scales space.

- In two dimensions, $|\det(A)|$ is the area scaling factor.
- In three dimensions, $|\det(A)|$ is the volume scaling factor.

If

$$\det(A) = 0,$$

then the transformation compresses the space into a lower dimension (such as a line or a plane), making the matrix non-invertible.

23.2 Sign of the Determinant

- $\det(A) > 0$: Orientation is preserved.
- $\det(A) < 0$: Orientation is reversed (reflection).

24 Example of Determinant

For

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

$$\boxed{\det(A) = ad - bc.}$$

25 Geometric Interpretation of Linear Transformation

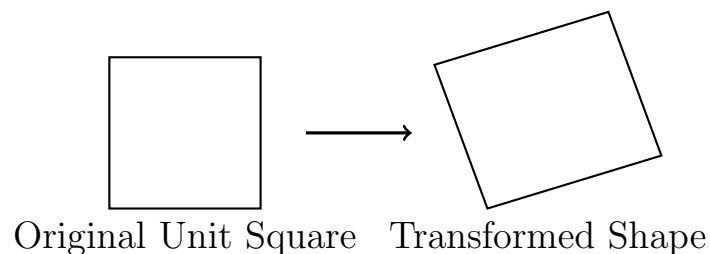
Consider the unit square with vertices

$$(0, 0), (1, 0), (0, 1), (1, 1).$$

Applying the matrix A transforms every point of the square.

- The square becomes a parallelogram.
- The area may increase or decrease.
- The orientation may also flip.
- The area scaling factor is

$$|\det(A)|.$$



26 Solving Linear Systems

Consider the system

$$Ax = b.$$

If A is invertible, then

$$\boxed{x = A^{-1}b.}$$

The inverse transformation maps the transformed vector b back to the original vector x .

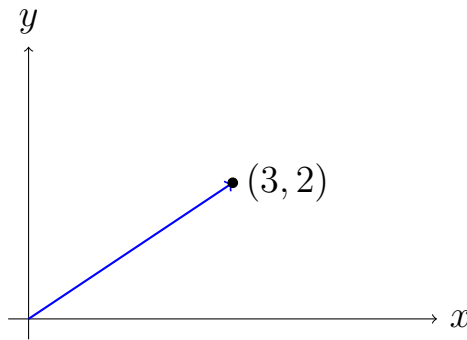
27 Example of a Vector

Consider the vector

$$v = 3\hat{i} + 2\hat{j}.$$

It can also be written as

$$v = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$



28 Linear Transformation

A matrix A defines a linear transformation

$$A : \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

which maps a vector x to another vector

$$Ax.$$

29 Eigenvalue Problem

Suppose we wish to find vectors whose direction remains unchanged after transformation.

These vectors satisfy

$$Ax = \lambda x,$$

where

- x is an eigenvector,

- λ is the corresponding eigenvalue.

Rearranging,

$$Ax - \lambda x = 0,$$

$$(A - \lambda I)x = 0.$$

For a non-zero solution,

$$\boxed{\det(A - \lambda I) = 0.}$$

This equation is called the **characteristic equation**.

30 Interpretation of Eigenvectors

Normally, a transformation changes both the magnitude and direction of a vector.

However, an eigenvector changes only its magnitude while preserving its direction.

Mathematically,

$$Ax = \lambda x.$$

If

$$\lambda > 1,$$

the vector is stretched.

If

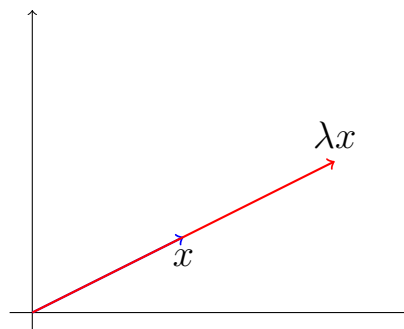
$$0 < \lambda < 1,$$

the vector shrinks.

If

$$\lambda < 0,$$

the direction is reversed while being scaled by $|\lambda|$.



31 Interpretation of Eigenvalues and Eigenvectors

Recall the eigenvalue equation

$$Ax = \lambda x,$$

where A is a transformation matrix, x is an eigenvector, and λ is the corresponding eigenvalue.

The matrix transforms every vector in space, but eigenvectors are special because their direction remains unchanged after the transformation. They are only stretched, compressed, or flipped.

- If $|\lambda| > 1$, the eigenvector is stretched.
- If $0 < |\lambda| < 1$, the eigenvector is compressed.
- If $\lambda < 0$, the vector reverses its direction (rotates by 180°) while being scaled.

32 Example

Consider the matrix

$$A = \begin{bmatrix} 0.8 & 0.6 \\ 0.2 & 0.4 \end{bmatrix}.$$

Suppose

$$u_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad u_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Verify whether they are eigenvectors.

32.1 Verification of u_1

$$Au_1 = \begin{bmatrix} 0.8 & 0.6 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2.4 + 0.6 \\ 0.6 + 0.4 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} = 1 \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Hence,

$$\boxed{\lambda_1 = 1.}$$

32.2 Verification of u_2

$$Au_2 = \begin{bmatrix} 0.8 & 0.6 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.8 + 0.6 \\ -0.2 + 0.4 \end{bmatrix} = \begin{bmatrix} -0.2 \\ 0.2 \end{bmatrix} = 0.2 \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Therefore,

$$\boxed{\lambda_2 = 0.2.}$$

33 Finding the Eigenvalues

The eigenvalues satisfy

$$Ax = \lambda x.$$

Rearranging,

$$(A - \lambda I)x = 0.$$

For a non-trivial solution,

$$\boxed{\det(A - \lambda I) = 0.}$$

For the matrix

$$A = \begin{bmatrix} 0.8 & 0.6 \\ 0.2 & 0.4 \end{bmatrix},$$

we obtain

$$\begin{vmatrix} 0.8 - \lambda & 0.6 \\ 0.2 & 0.4 - \lambda \end{vmatrix} = 0.$$

Expanding,

$$(0.8 - \lambda)(0.4 - \lambda) - 0.12 = 0.$$

$$0.32 - 1.2\lambda + \lambda^2 - 0.12 = 0.$$

$$\lambda^2 - 1.2\lambda + 0.20 = 0.$$

Applying the quadratic formula,

$$\begin{aligned} \lambda &= \frac{1.2 \pm \sqrt{1.2^2 - 4(1)(0.2)}}{2} \\ &= \frac{1.2 \pm \sqrt{1.44 - 0.80}}{2} = \frac{1.2 \pm 0.8}{2}. \end{aligned}$$

Hence,

$$\boxed{\lambda_1 = 1, \quad \lambda_2 = 0.2.}$$

34 Finding the Eigenvectors

34.1 Eigenvector Corresponding to $\lambda = 1$

Solve

$$(A - I)x = 0.$$

$$\begin{bmatrix} -0.2 & 0.6 \\ 0.2 & -0.6 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0.$$

This gives

$$-0.2x + 0.6y = 0.$$

or

$$x = 3y.$$

Choosing

$$y = 1,$$

we obtain

$$u_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

34.2 Eigenvector Corresponding to $\lambda = 0.2$

Solve

$$(A - 0.2I)x = 0.$$

$$\begin{bmatrix} 0.6 & 0.6 \\ 0.2 & 0.2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0.$$

From the first equation,

$$0.6x + 0.6y = 0,$$

or

$$y = -x.$$

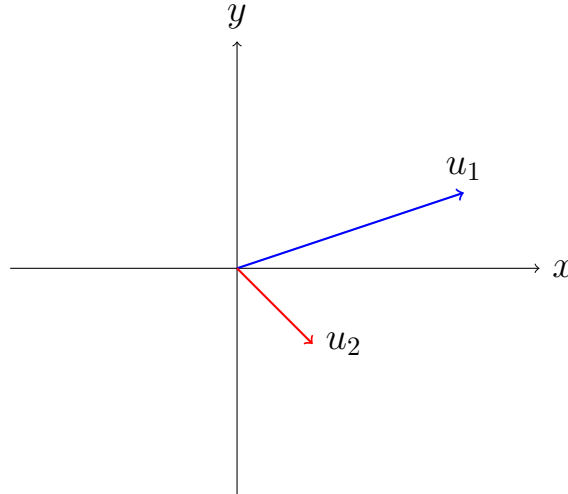
Taking

$$x = 1,$$

we obtain

$$u_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

35 Geometric Illustration



The blue vector remains unchanged because its eigenvalue is 1, while the red vector is compressed since its eigenvalue is 0.2.

36 Practice Problems

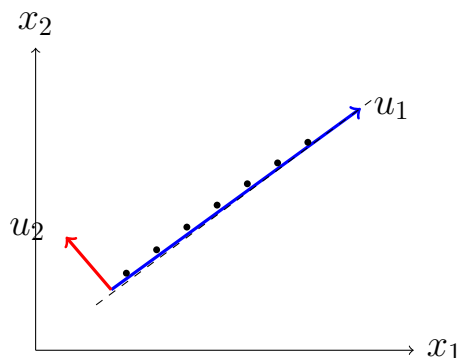
Find the eigenvalues and eigenvectors of the following matrix:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}.$$

1. Find the eigenvalues of A .
2. Find the corresponding eigenvectors for each eigenvalue.

37 Principal Component Analysis (PCA): Motivation

Suppose our data consists of two features, x_1 and x_2 . The observations are highly concentrated along one direction while exhibiting only a very small variation in the perpendicular direction.



The figure shows that most observations lie close to the direction u_1 , whereas the spread along u_2 is very small.

- Therefore, the data can be represented approximately in a lower-dimensional space.
- Instead of using the original two-dimensional space $\mathbb{R}^2$, we can project the data onto the principal direction u_1 with only a small loss of information.
- The variance along u_1 is much larger than the variance along u_2 .

Consequently, PCA keeps the directions with the largest variance while discarding directions with very small variance.

38 Why is the Second Direction Less Important?

The variance of the data along u_2 is very small. Therefore,

- points become difficult to distinguish along u_2 ,
- very little new information is contained in this direction,
- removing this direction results in only a small loss of information.

Hence, PCA projects the data onto the direction having maximum variance.

39 Correlation Between Features

Consider the following observations.

x	y	z
1.00	1.00	1.00
0.50	0.00	0.00
0.25	1.00	1.00
0.35	1.50	1.50
0.45	1.00	1.00
0.57	2.00	2.10
0.62	1.10	1.00
0.73	0.75	0.76
0.72	0.86	0.87

From the observations, it is evident that variables y and z are highly correlated. The Pearson correlation coefficient between y and z is

$$\rho_{yz} = \frac{\sum_{i=1}^n (y_i - \bar{y})(z_i - \bar{z})}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2} \sqrt{\sum_{i=1}^n (z_i - \bar{z})^2}}.$$

If

$$\rho_{yz} \approx 1,$$

then the two variables contain almost the same information. PCA exploits this redundancy by combining correlated variables into a smaller number of principal components.

40 Centering the Data

Before applying PCA, the data must be centered.

For each feature,

$$x'_i = x_i - \bar{x},$$

where

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Similarly,

$$y'_i = y_i - \bar{y}, \quad z'_i = z_i - \bar{z}.$$

After centering,

Each feature has zero mean.

This step ensures that PCA captures the directions of maximum variance rather than being influenced by the absolute values of the features.

41 Key Observations

- PCA represents high-dimensional data using fewer dimensions.
- The first principal component captures the maximum variance.
- Subsequent principal components capture the remaining variance while remaining orthogonal to previous components.

- Highly correlated variables contribute similar information and can often be represented by fewer principal components.
- Data centering (zero-mean transformation) is an essential preprocessing step before computing principal components.

42 Representation of Data in a New Orthonormal Basis

Let

$$P_1, P_2, \dots, P_n$$

be a set of n orthonormal and linearly independent vectors. Construct the matrix

$$P = \begin{bmatrix} | & | & \cdots & | \\ P_1 & P_2 & \cdots & P_n \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{n \times n},$$

whose columns are the basis vectors.

Suppose

$$x_1, x_2, \dots, x_m \in \mathbb{R}^n$$

are the m data samples, and define the data matrix

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix},$$

where each row represents one observation. Assume that the data has already been centered (zero mean) and normalized to unit variance.

Our objective is to represent each sample using the new orthonormal basis.

For any sample x_i ,

$$x_i = \alpha_{i1}P_1 + \alpha_{i2}P_2 + \cdots + \alpha_{in}P_n.$$

Since the basis vectors are orthonormal,

$$\boxed{\alpha_{ij} = x_i^T P_j.}$$

Collecting all coefficients,

$$\alpha_i = \begin{bmatrix} \alpha_{i1} \\ \alpha_{i2} \\ \vdots \\ \alpha_{in} \end{bmatrix} = P^T x_i.$$

Hence, the transformed representation of every sample is obtained by

$$\hat{x}_i = P^T x_i.$$

Stacking all transformed samples together,

$$\boxed{\hat{X} = X P,}$$

where

$$\hat{X} = \begin{bmatrix} \hat{x}_1^T \\ \hat{x}_2^T \\ \vdots \\ \hat{x}_m^T \end{bmatrix}.$$

Thus, the transformed data matrix contains the coordinates of every sample in the new orthonormal basis.

43 Covariance Matrix of the Transformed Data

Assume that the columns of X are zero-mean. The covariance matrix of the original data is

$$\boxed{\Sigma = \frac{1}{m} X^T X.}$$

Each element of the covariance matrix represents the covariance between two features.

Let $C = [C_{ij}]$ denote the covariance matrix. By definition,

$$C_{ij} = \frac{1}{m} \sum_{k=1}^m (x_{ki} - \mu_i)(x_{kj} - \mu_j),$$

where μ_i and μ_j are the means of the i^{th} and j^{th} features, respectively. Since the data is centered,

$$\mu_i = \mu_j = 0,$$

which simplifies the covariance to

$$C_{ij} = \frac{1}{m} \sum_{k=1}^m x_{ki} x_{kj}.$$

This can be written compactly as

$$C_{ij} = \frac{1}{m} (X^T X)_{ij}.$$

Therefore,

$$\Sigma = \frac{1}{m} X^T X.$$

44 Covariance of the Transformed Data

The transformed data is

$$\hat{X} = XP.$$

Its covariance matrix is

$$\frac{1}{m} \hat{X}^T \hat{X}.$$

Substituting $\hat{X} = XP$,

$$\begin{aligned} \frac{1}{m} \hat{X}^T \hat{X} &= \frac{1}{m} (XP)^T (XP) \\ &= \frac{1}{m} P^T X^T X P \\ &= P^T \left(\frac{1}{m} X^T X \right) P. \end{aligned}$$

Since

$$\Sigma = \frac{1}{m} X^T X,$$

the covariance matrix in the transformed coordinate system becomes

$$\hat{\Sigma} = P^T \Sigma P.$$

This equation forms the mathematical basis of Principal Component Analysis (PCA). By choosing the columns of P as the eigenvectors of Σ , the transformed covariance matrix becomes diagonal, indicating that the principal components are uncorrelated.

45 Principal Component Analysis (PCA)

Recall that the covariance matrix of the transformed data is

$$\hat{\Sigma} = P^T \Sigma P.$$

Our objective is to find a basis in which the covariance matrix becomes diagonal. Specifically, we require

$$\left(\frac{1}{m} \hat{X}^T \hat{X} \right)_{ij} = 0, \quad i \neq j,$$

that is, all off-diagonal entries (covariances) are zero.
For the diagonal entries,

$$\left(\frac{1}{m}\hat{X}^T\hat{X}\right)_{ii} \neq 0,$$

since they represent the variances of the transformed variables.
Hence, we want

$$\boxed{\frac{1}{m}\hat{X}^T\hat{X} = P^T\Sigma P = D,}$$

where D is a diagonal matrix.
Since Σ is a real symmetric matrix and P is orthogonal,

$$P^TP = PP^T = I.$$

Therefore, the desired condition is

$$\boxed{P^T\Sigma P = D.}$$

The solution is obtained by choosing the columns of P as the eigenvectors of the covariance matrix,

$$\Sigma = \frac{1}{m}X^TX.$$

Thus,

$$P = \begin{bmatrix} | & | & & | \\ p_1 & p_2 & \cdots & p_n \\ | & | & & | \end{bmatrix},$$

where

$$\Sigma p_i = \lambda_i p_i.$$

The eigenvectors form an orthonormal basis, and transforming the data into this basis decorrelates the features.

46 Dimensionality Reduction

In practice, we retain only the first k principal components corresponding to the largest eigenvalues.

The approximation of the i^{th} sample is

$$\boxed{\hat{x}_i = \sum_{j=1}^k \alpha_{ij} p_j.}$$

47 Reconstruction Error Minimization in PCA

We minimize the reconstruction error

$$e^2 = \sum_{i=1}^m (x_i - \hat{x}_i)^2.$$

Since

$$x_i = \sum_{j=1}^n \alpha_{ij} p_j, \quad \hat{x}_i = \sum_{j=1}^k \alpha_{ij} p_j,$$

we obtain

$$\begin{aligned} e^2 &= \sum_{i=1}^m \left(\sum_{j=1}^n \alpha_{ij} p_j - \sum_{j=1}^k \alpha_{ij} p_j \right)^2 \\ &= \sum_{i=1}^m \left(\sum_{j=k+1}^n \alpha_{ij} p_j \right)^2. \end{aligned}$$

Expanding,

$$\begin{aligned} e^2 &= \sum_{i=1}^m \left(\sum_{j=k+1}^n \alpha_{ij} p_j \right)^T \left(\sum_{l=k+1}^n \alpha_{il} p_l \right) \\ &= \sum_{i=1}^m \sum_{j=k+1}^n \sum_{l=k+1}^n \alpha_{ij} p_j^T p_l \alpha_{il}. \end{aligned}$$

Using

$$p_j^T p_j = 1, \quad p_i^T p_j = 0, \quad i \neq j,$$

we get

$$e^2 = \sum_{i=1}^m \sum_{j=k+1}^n \alpha_{ij}^2 = \sum_{j=k+1}^n \sum_{i=1}^m \alpha_{ij}^2.$$

48 Expressing the Error in Matrix Form

Since

$$\alpha_{ij} = p_j^T x_i,$$

therefore,

$$\begin{aligned}
e^2 &= \sum_{j=k+1}^n \sum_{i=1}^m (p_j^T x_i) (x_i^T p_j) \\
&= \sum_{j=k+1}^n p_j^T \left(\sum_{i=1}^m x_i x_i^T \right) p_j.
\end{aligned}$$

Since

$$C = \frac{1}{m} \sum_{i=1}^m x_i x_i^T,$$

we have

$$\sum_{i=1}^m x_i x_i^T = mC.$$

Hence,

$$e^2 = \sum_{j=k+1}^n p_j^T (mC) p_j.$$

49 Optimization Problem

We want to minimize

$$\min_{P_{k+1}, \dots, P_n} \sum_{j=k+1}^n p_j^T (mC) p_j$$

subject to

$$p_j^T p_j = 1, \quad j = k+1, \dots, n.$$

50 Solution

The solution of the above optimization problem is obtained by choosing

$$p_{k+1}, p_{k+2}, \dots, p_n$$

as the eigenvectors corresponding to the smallest eigenvalues of the covariance matrix C .

Thus,

$$P = [p_1, p_2, \dots, p_n],$$

where the columns are eigenvectors of C .

For dimensionality reduction, we retain only the first k eigenvectors corresponding to the largest eigenvalues.

51 Variance Maximization Interpretation

We want to maximize

$$\sum_{i=1}^m (p_j^T x_i)^2.$$

Using matrix notation,

$$\begin{aligned} \sum_{i=1}^m (p_j^T x_i)^2 &= \sum_{i=1}^m (p_j^T x_i) (x_i^T p_j) \\ &= p_j^T \left(\sum_{i=1}^m x_i x_i^T \right) p_j \\ &= p_j^T (mC) p_j. \end{aligned}$$

Hence,

$$\boxed{\max_{p_j} p_j^T (mC) p_j, \quad \text{subject to } p_j^T p_j = 1.}$$

This optimization is known as the Rayleigh Quotient.
Its solution is

$$Cp_j = \lambda_j p_j,$$

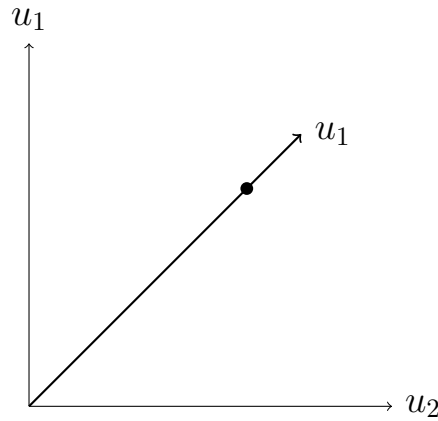
where p_j is an eigenvector of the covariance matrix C .

Therefore,

- the first principal component is the eigenvector corresponding to the largest eigenvalue,
- the second principal component corresponds to the second largest eigenvalue,
- and so on.

Thus, PCA preserves the directions with maximum variance while minimizing reconstruction error.

52 Example: PCA Reconstruction



Suppose

$$u_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad u_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

These are the two basis vectors.

To convert them into unit vectors,

$$u_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad u_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Consider the point

$$x = \begin{bmatrix} 3 \\ 3 \end{bmatrix}.$$

The coordinates of x in the new basis are

$$\alpha_1 = x^T u_1 = \frac{6}{\sqrt{2}} = 3\sqrt{2},$$

and

$$\alpha_2 = x^T u_2 = 0.$$

Hence,

$$x = \alpha_1 u_1 + \alpha_2 u_2.$$

Substituting the values,

$$x = 3\sqrt{2} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) + 0 \left(\frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \right),$$

which gives

$$x = \begin{bmatrix} 3 \\ 3 \end{bmatrix}.$$

Thus, the projection perfectly reconstructs the original vector since the second coefficient is zero.

If we retain only the first principal component ($k = 1$), ignoring the low-variance direction,

$$\hat{x} = \alpha_1 u_1 = 3\sqrt{2} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \boxed{\begin{bmatrix} 3 \\ 3 \end{bmatrix}}.$$

Therefore, the reconstruction error is zero because the data lies entirely along the first principal component.

53 Singular Value Decomposition (SVD)

Consider the vector

$$x = \begin{bmatrix} 1 \\ 3 \end{bmatrix},$$

and the matrix

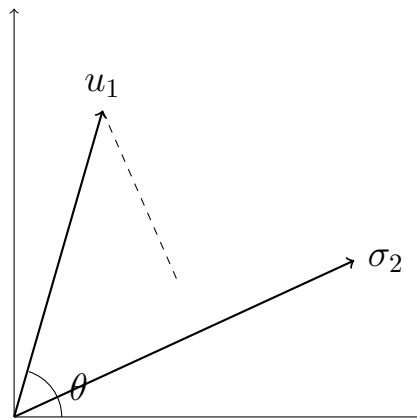
$$A = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}.$$

Then

$$Ax = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}.$$

Thus, the transformation A performs a combination of rotation and stretching.

53.1 Geometric Interpretation



53.2 Rotation Matrix

A rotation matrix is

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix},$$

which rotates a vector by an angle θ .

53.3 Stretching Matrix

The stretching (scaling) matrix is

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix},$$

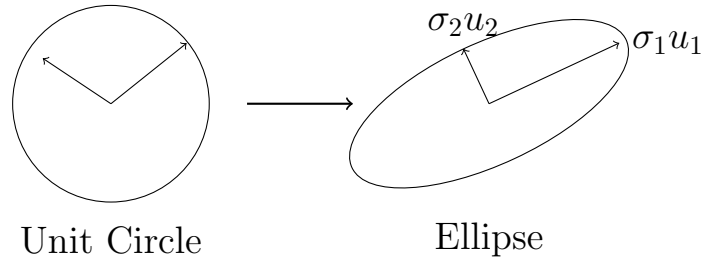
where

$$\sigma_1, \sigma_2$$

represent the amount of stretching along the principal directions.

54 Geometric Meaning of SVD

Consider the unit circle in $\mathbb{R}^2$.



When a matrix

$$A \in \mathbb{R}^{m \times n}$$

acts on the unit circle, the image becomes an ellipse.

The principal axes of the ellipse are

$$u_1, u_2, \dots, u_r,$$

called the left singular vectors.

The lengths of the semi-major and semi-minor axes are

$$\sigma_1, \sigma_2, \dots, \sigma_r,$$

called the singular values.

Thus,

$$A = U\Sigma V^T,$$

where

- V : rotates the input vectors,
- Σ : stretches or compresses them,
- U : rotates the transformed vectors to the final orientation.

54.1 Hyperplane Interpretation

For a general matrix

$$A \in \mathbb{R}^{m \times n},$$

the transformation maps vectors from an n -dimensional space into an m -dimensional space.

The image of the transformation lies in a lower-dimensional subspace (hyperplane) whenever the rank of the matrix is less than the ambient dimension.

54.2 Singular Vectors

The right singular vectors are

$$v_1, v_2, \dots, v_n,$$

while the left singular vectors are

$$u_1, u_2, \dots, u_m.$$

The singular values satisfy

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0.$$

Each singular value represents the amount of stretching along the corresponding singular vector.

Hence,

$$Av_i = \sigma_i u_i.$$

The largest singular value corresponds to the direction of maximum stretching, whereas the smallest singular value corresponds to the direction of minimum stretching.

From the definition,

$$Av_i = \sigma_i u_i, \quad i = 1, 2, \dots, n.$$

Hence,

$$A \begin{bmatrix} v_1 & v_2 & \dots & v_n \end{bmatrix} = \begin{bmatrix} \sigma_1 u_1 & \sigma_2 u_2 & \dots & \sigma_n u_n \end{bmatrix}.$$

Since

$$\begin{bmatrix} \sigma_1 u_1 & \sigma_2 u_2 & \dots & \sigma_n u_n \end{bmatrix} = \begin{bmatrix} u_1 & u_2 & \dots & u_n \end{bmatrix} \begin{bmatrix} \sigma_1 & & & 0 \\ & \sigma_2 & & \\ & & \ddots & \\ 0 & & & \sigma_n \end{bmatrix},$$

we obtain

$$A \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix} = U\Sigma.$$

Multiplying both sides by V^T ,

$$AV = U\Sigma,$$

$$AVV^T = U\Sigma V^T,$$

Since

$$VV^T = I,$$

we finally get

$$\boxed{A = U\Sigma V^T.}$$

Thus, any matrix can be decomposed into

$$\boxed{A = U\Sigma V^T,}$$

where

- U : Left singular vectors - Σ : Singular value matrix - V : Right singular vectors

This decomposition tells us that every matrix transformation can be represented as

1. Rotation, 2. Stretching, 3. Rotation.

Hence, instead of applying one complicated transformation, we apply three simple orthogonal transformations.

The unitary (orthogonal) matrices preserve lengths and angles in the input space.

55 Compact Singular Value Decomposition

Suppose

$$A \in \mathbb{R}^{m \times n}.$$

The complete SVD is

$$A = U\Sigma V^T.$$

Dimension-wise,

$$\underbrace{A}_{m \times n} = \underbrace{U}_{m \times m} \underbrace{\Sigma}_{m \times n} \underbrace{V^T}_{n \times n}.$$

Only

$$r = \text{rank}(A)$$

diagonal entries of Σ are non-zero.

Therefore, we keep only those singular vectors corresponding to the non-zero singular values.

The compact SVD becomes

$$\underbrace{A}_{m \times n} = \underbrace{U_r}_{m \times r} \underbrace{\Sigma_r}_{r \times r} \underbrace{V_r^T}_{r \times n}.$$

where

$$r = \text{rank}(A).$$

The rank of a matrix is the maximum number of linearly independent rows (or columns).

55.1 Properties of Singular Values

The singular values satisfy

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0.$$

Thus, every singular value is either positive or zero.

56 Theorem

Every matrix

$$A \in \mathbb{R}^{m \times n}$$

has a Singular Value Decomposition,

$$\boxed{A = U \Sigma V^T.}$$

56.1 Properties

The singular values

$$\{\sigma_i\}$$

are uniquely determined.

If all singular values are distinct,
then the singular vectors

$$\{u_i\}, \quad \{v_i\}$$

are unique up to sign.

57 Relation with Eigenvalues

Starting from

$$A = U\Sigma V^T,$$

compute

$$A^T A.$$

Substituting SVD,

$$A^T A = (U\Sigma V^T)^T (U\Sigma V^T).$$

Taking transpose,

$$A^T A = V\Sigma^T U^T U\Sigma V^T.$$

Since

$$U^T U = I,$$

we obtain

$$A^T A = V\Sigma^T \Sigma V^T.$$

Multiplying by V ,

$$A^T AV = V\Sigma^T \Sigma.$$

Hence,

$$\boxed{A^T AV = V\Sigma^T \Sigma.}$$

Therefore,

- the eigenvectors of $A^T A$ are the columns of V , - the eigenvalues of $A^T A$ are

$$\boxed{\lambda_i = \sigma_i^2.}$$

58 Finding the Left Singular Vectors

Similarly,

$$AA^T = (U\Sigma V^T)(U\Sigma V^T)^T.$$

Taking transpose,

$$AA^T = U\Sigma V^T V\Sigma^T U^T.$$

Since

$$V^T V = I,$$

we have

$$AA^T = U\Sigma\Sigma^T U^T.$$

Hence,

$$AA^T U = U\Sigma\Sigma^T.$$

Therefore,

$$\boxed{AA^T U = U\Sigma\Sigma^T.}$$

Thus,

- the columns of U are the eigenvectors of AA^T , - the eigenvalues are again

$$\sigma_i^2.$$

Therefore,

$$\boxed{\begin{array}{l} A^T A \longrightarrow \text{Right Singular Vectors } (V), \\ AA^T \longrightarrow \text{Left Singular Vectors } (U). \end{array}}$$